



**МИНОБРНАУКИ РОССИИ**  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
**«МИРЭА - Российский технологический университет»**  
**РТУ МИРЭА**

---

**Институт Информационных Технологий**  
**Кафедра Вычислительной Техники**

**ПРАКТИЧЕСКАЯ РАБОТА №3**  
  
**по дисциплине**  
**«Разработка обеспечивающих подсистем систем поддержки**  
**принятия решений»**

Студент группы: ИКБО-04-22

Кликушин В.И.  
(Ф. И.О. студента)

Преподаватель

Гуличева А.А.  
(Ф.И.О. преподавателя)

Москва 2025

# СОДЕРЖАНИЕ

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                                  | 3  |
| 1 ПОСТАНОВКА ЗАДАЧИ .....                                                       | 5  |
| 2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ .....                                                     | 6  |
| 2.1 Сингулярное разложение .....                                                | 6  |
| 2.2 Алгоритм конструирования сингулярного разложения произвольной матрицы ..... | 9  |
| 2.3 Измерение качества рекомендаций .....                                       | 10 |
| 3 ДОКУМЕНТАЦИЯ К ДАННЫМ .....                                                   | 12 |
| 3.1 Описание предметной области .....                                           | 12 |
| 3.2 Анализ данных .....                                                         | 12 |
| 3.3 Предобработка данных .....                                                  | 16 |
| 4 ПРАКТИЧЕСКАЯ ЧАСТЬ .....                                                      | 17 |
| 4.1 Функциональные возможности .....                                            | 17 |
| 4.2 Сингулярное разложение .....                                                | 21 |
| ЗАКЛЮЧЕНИЕ .....                                                                | 25 |
| СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ .....                                          | 26 |
| ПРИЛОЖЕНИЯ .....                                                                | 27 |

# ВВЕДЕНИЕ

Современные стриминговые платформы и онлайн-сервисы ежедневно сталкиваются с проблемой эффективного управления вниманием пользователей в условиях постоянно растущего объема доступного контента. С расширением каталогов до десятков тысяч фильмов традиционные методы ручного поиска становятся недостаточными для удовлетворения индивидуальных предпочтений. Проблема информационной перегрузки особенно остро стоит в сфере развлекательного контента, где пользователи тратят значительное время на поиск релевантных фильмов вместо непосредственного просмотра.

В области рекомендательных систем традиционно доминируют методы коллаборативной фильтрации, основанные на поиске схожих пользователей (user-based) или объектов (item-based). Однако эти методы имеют фундаментальные ограничения — они плохо масштабируются на большие объемы данных и сталкиваются с проблемой «холодного старта» для новых пользователей или контента. Кроме того, они не учитывают скрытые закономерности в предпочтениях пользователей.

Перспективным направлением являются модельные методы (model-based), в частности методы матричной факторизации, такие как сингулярное разложение. Эти подходы позволяют выявлять скрытые факторы, объясняющие пользовательские предпочтения, и эффективно работать с разреженными матрицами оценок.

В данной работе реализована комплексная рекомендательная система на основе сингулярного разложения матрицы оценок, дополненная методами кластерного анализа для группировки пользователей с похожими предпочтениями. Для экспериментальной проверки эффективности подхода выбран датасет MovieLens, содержащий реальные оценки пользователей с временными метками.

Структура работы включает: анализ предметной области и данных, теоретическое обоснование сингулярного разложения и кластерного анализа,

практическую реализацию консольной системы с поддержкой различных стратегий рекомендаций, а также оценку эффективности предложенного подхода с использованием метрик MSE, RMSE и MAE.

Практическая значимость работы заключается в разработке прототипа системы, способной эффективно обрабатывать большие объемы данных и предоставлять персонализированные рекомендации на основе выявления скрытых паттернов в пользовательских предпочтениях, что может быть использовано для улучшения рекомендательных механизмов в реальных стриминговых платформах и маркетплейсах.

# 1 ПОСТАНОВКА ЗАДАЧИ

Цель работы: приобрести навыки реализации модельных коллаборативных методов рекомендательных систем, основанных на поиске скрытых факторов методами сингулярного разложения и кластерного анализа.

Задачи: создать программную реализацию модельного метода рекомендательной системы (РС), основанную на сингулярном разложении и кластерном анализе и включающую актуальную предметную область для применения РС (вроде маркетплейса, медиа ресурсов, соц. сетей, экономической сферы и т.д.) и набор начальных данных для неё, привести входные данные к матричному виду для выполнения её сингулярного разложения вида  $A = UDV^T$  и получения на его основе предпочтений пользователей, характеристики объектов и сингулярных значений как значимость воздействия факторов на пользователей, интерпретировать эти факторы для человеческого восприятия, произвести измерение качества рекомендаций на основе метрик качества (вроде MSE, MAE и т.д.) и реальных оценок полученных пользователями рекомендаций, выполнить кластеризацию пользователей или их рекомендаций (результатов предыдущих пунктов задания) с помощью EM-алгоритма или иерархической кластеризации, реализовать алгоритм Funk SVD с минимизацией найденной ошибки для работа с явными данными пользователей и алгоритма SVD++ с проведением регуляризации для компенсации переобучения для работа с неявными данными, выполнить оба алгоритма кластерного анализа, приведённых выше, и реализовать для них несколько методов измерения расстояния между кластерами.

## **2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ**

Несмотря на легкость понимания и реализации анамнестических методов, у них есть ряд недостатков. Во-первых, возникают трудности при прогнозе предпочтений для новых пользователей или при появлении новых элементов, т. к. для них еще нет оценок («холодный старт»). Во-вторых, для расчета предпочтений необходимо хранить всю матрицу данных. В-третьих, ограничивается возможность методов при обработке больших объемов данных, т. к. выполнение большого количества операций для вычисления степени схожести затрудняет выдачу рекомендаций в реальном времени. Большой объем матрицы предпочтений затрагивает также проблему избыточности данных. Как правило, пользователи и элементы делятся на группы с аналогичными профилями предпочтений.

Вследствие этого возникает задача в понижении размерности матрицы оценок. Такие задачи решают методы второй группы модельные методы (Model-based). Эти методы направлены на поиск закономерностей на основе обучающих данных. Такой подход является комплексным и даёт более точные прогнозы, так как помогает раскрыть скрытые факторы, объясняющие наблюдаемые оценки.

В этом случае возможен вариант объединения пользователей (элементов) в кластеры (профили) с помощью некоторого индекса сходства. Элементы и оценки, выставленные пользователями из одного кластера, будут использоваться для вычисления рекомендаций. Кластерные модели лучше масштабируются, т. к. сверяют профиль пользователя с относительно небольшим количеством сегментов, а не с целой пользовательской базой.

### **2.1 Сингулярное разложение**

Для уменьшения сложности вычисления степени схожести векторов объектов используется подход понижения размерности матрицы, который

основан на разложении матрицы по сингулярным значениям. Это один из методов матричной факторизации.

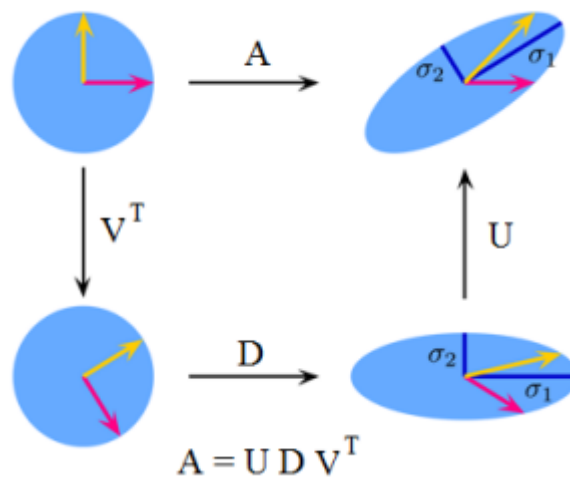
Суть метода сингулярного разложения матрицы (Singular Value Decomposition, SVD)  $A \in M(n, m)$  с рангом  $d = \text{rank}(M) \leq \min(n, m)$  в произведение матриц меньшего ранга (Формула 2.1).

$$A = UDV^T \quad (2.1)$$

где матрицы  $U \in M(n, n)$  и  $V \in M(m, m)$  состоят из ортонормальных столбцов, являющихся собственными векторами при ненулевых собственных значениях матрицы  $AA^T$  и  $A^T A$ .

Ортогональная матрица – квадратная матрица  $A$  с вещественными элементами, результат умножения которой на транспонированную матрицу  $A^T$  равен единичной матрице:  $AA^T = A^T A = I$ .

Формулу сингулярного разложения  $A = UDV^T$  можно переформулировать в геометрических терминах (Рисунок 2.1).



**Рисунок 2.1 - Геометрический смысл сингулярного разложения**

Вверху слева (Рисунок 2.1) видно, единичный диск (синий цвет) с двумя каноническими единичными векторами. Действие  $V^T$  (внизу слева) на единичном диске — это просто вращение (против часовой стрелки) на угол  $\alpha$ . Действие  $D$  (справа внизу) масштабирует единичный круг по вертикали и

горизонталю (вдоль осей координат). Действие  $U$  приводит к окончательному повороту с указанными сингулярными значениями  $\sigma_1$  и  $\sigma_2$ . Длины  $\sigma_1$  и  $\sigma_2$  из полуосей эллипса являются сингулярными значениями из  $A$ .

Соответственно, справедливы следующие выражения (Формула 2.2).

$$U * U^T = I_n; V * V^T = I_m \quad (2.2)$$

Столбцы матрицы  $U$  представляют собой, ортонормальный базис пространства столбцов матрицы  $A$ , а столбцы матрицы  $V$  – ортонормальный базис пространства строк матрицы  $A$ .  $I_n$  и  $I_m$  – единичные вектора.  $D \in M(n, m)$  – диагональная матрица с положительными диагональными элементами  $\sigma_1 \geq \sigma_2 \dots \geq \sigma_d \geq 0$ , отсортированного в порядке убывания (Рисунок 2.2).

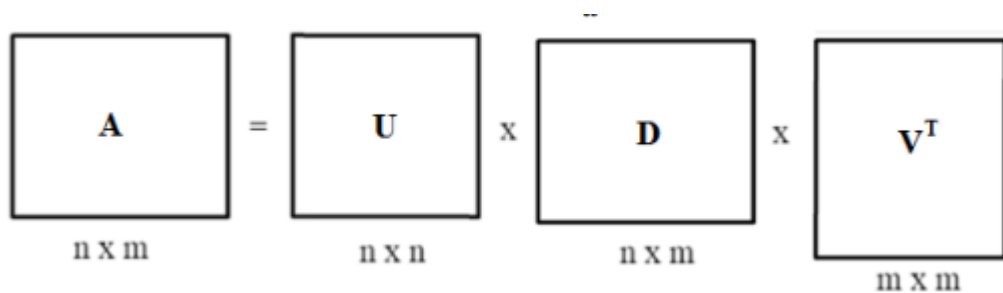


Рисунок 2.2 – Сингулярное разложение

Векторно-матричное выражение (Формула 2.3) представляет собой решение задачи нахождения собственных чисел  $\sigma_i^2$  и собственных векторов  $U_i$  матрицы  $AA^T$ .

$$D = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \dots & 0 \\ 0 & 0 & \sigma_d \end{pmatrix} \quad (2.3)$$

В результате получаем, что сингулярные числа  $\sigma_i^2$  ( $i = 1, \dots, m$ ) представляют собой решения характеристического уравнения (Формула 2.4).

$$\det(\sigma_i^2 I - AA^T) = 0 \quad (2.4)$$



Также можно получить зеркальные соотношения (Формула 2.5).

$$A^T A V_i = \sigma_i^2 V_i; i = 1, \dots, m \quad (2.5)$$

где  $V_i$  – столбцы матрицы  $V$ ;

$\sigma_i^2$  – квадраты соответствующих значений на диагонали матрицы  $D$ .

В свою очередь векторно-матричное выражение (Формула 2.5) представляет собой полное решение задачи нахождения собственных чисел  $\sigma_i^2$  и собственных векторов  $V_i$  матрицы  $A^T A$ . В результате получаем, что  $\sigma_i^2$  ( $i = 1, \dots, m$ ) представляют собой решения характеристического уравнения (Формула 2.6).

$$\det(\sigma_i^2 I - A^T A) = 0 \quad (2.6)$$

## 2.2 Алгоритм конструирования сингулярного разложения произвольной матрицы

При конструировании сингулярного разложения произвольной  $(n, m)$  - матрицы  $A$  ранга  $d$  следует придерживаться следующего порядка действий.

1. Составить матрицу  $A^T A$  и характеристический полином этой матрицы. Найти сингулярные числа матрицы  $A$  и составить  $(n, m)$  - матрицу  $D$ .
2. Построить ортонормированную систему собственных векторов  $e_1, e_2, \dots, e_n$  оператора с матрицей  $A^T A$ , принадлежащих соответственно собственным значениям  $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ . Из столбцов координат этих векторов составить матрицу  $V$ .
3. Построить ортонормированную систему векторов (Формула 2.7).

$$f_1 = \frac{Ae_1}{\sigma_1}, \dots, f_r = \frac{Ae_r}{\sigma_r} \quad (2.7)$$

и дополнить ее любыми векторами до ортонормированного базиса пространства. Из столбцов координат векторов составить матрицу  $U$ .

4. Записать искомое сингулярное разложение  $A = UDV^T$ .

### 2.3 Измерение качества рекомендаций

Существует достаточное количество метрик (оценка качества) для тестирования рекомендательных систем. На данный момент не существует единой метрики. Каждый тестировщик подбирает метрику под свои цели. Для тестирования качества прогнозирования оценок в рекомендательных системах используются метрики:

1. Средняя абсолютная ошибка (MAE – Mean Absolute Error) (Формула 2.8).

$$MAE = \frac{1}{|D|} \sum_{(u,i) \in D} |\bar{r}_{ui} - r_{ui}|, \quad (2.8)$$

где  $|D|$  – размер набора данных;

$\bar{r}_{ui}$  – предсказанная оценка пользователя  $u$  фильму  $i$ ;

$r_{ui}$  – действительная оценка, поставленная пользователем  $u$  фильму  $i$ .

2. Среднеквадратичная ошибка (MSE – Mean Squared Error) (Формула 2.9)

$$MSE = \frac{1}{|D|} \sum_{(u,i) \in D} (\bar{r}_{ui} - r_{ui})^2, \quad (2.9)$$

где  $|D|$  – размер набора данных;

$\bar{r}_{ui}$  – предсказанная оценка пользователя  $u$  фильму  $i$ ;

$r_{ui}$  – действительная оценка, поставленная пользователем  $u$  фильму  $i$ .

3. Корень среднеквадратичной ошибки (RMSE – Root Mean Squared Error) (Формула 2.10).

$$RMSE = \sqrt{\frac{1}{|D|} \sum_{(u,i) \in D} (\bar{r}_{ui} - r_{ui})^2}, \quad (2.10)$$

где  $|D|$  – размер набора данных;

$\bar{r}_{ui}$  – предсказанная оценка пользователя  $u$  фильму  $i$ ;

$r_{ui}$  – действительная оценка, поставленная пользователем  $u$  фильму  $i$ .

## 3 ДОКУМЕНТАЦИЯ К ДАННЫМ

### 3.1 Описание предметной области

Данные взяты из открытого сервиса MovieLens, предоставляемого исследовательской группой GroupLens Университета Миннесоты. Датасет ml-latest-small содержит информацию о рейтингах и тегах пользователей для фильмов.

Целью использования этих данных является построение рекомендательной системы фильмов, которая может предсказывать оценки для фильмов, не просмотренных пользователем, и рекомендовать новые фильмы на основе исторических оценок и тегов.

Датасет состоит из следующих сущностей:

- пользователи (User): анонимизированные идентификаторы пользователей, которые выставляли оценки фильмов и добавляли теги;
- фильмы (Movie): идентификаторы, название, жанры, год выпуска и ссылки на внешние источники (IMDb, TMDb);
- рейтинги (Rating): пользователь, фильм, оценка от 0.5 до 5 и временная метка;
- теги (Tag): пользователь, фильм, текст тега и временная метка.

### 3.2 Анализ данных

Датасет ml-latest-small представляет собой компактную выборку данных MovieLens, содержащую оценки пользователей и информацию о фильмах. Он включает чуть более ста тысяч рейтингов, чуть меньше десяти тысяч фильмов и около шести сотен пользователей, что делает его удобным для экспериментов и обучения рекомендательных систем без необходимости использования мощных вычислительных ресурсов.

Каждая запись о рейтинге состоит из идентификатора пользователя, идентификатора фильма, выставленной оценки и временной метки. Основная информация о данных в файле ratings.csv представлена на Рисунке 3.2.1.

```
><class 'pandas.core.frame.DataFrame'>
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   userId      100836 non-null  int64
1   movieId     100836 non-null  int64
2   rating      100836 non-null  float64
3   timestamp   100836 non-null  int64
dtypes: float64(1), int64(3)
memory usage: 3.1 MB
```

Рисунок 3.2.1 - Основная информация о данных в файле ratings.csv

Распределение выставленных оценок представлено на Рисунке 3.2.2.

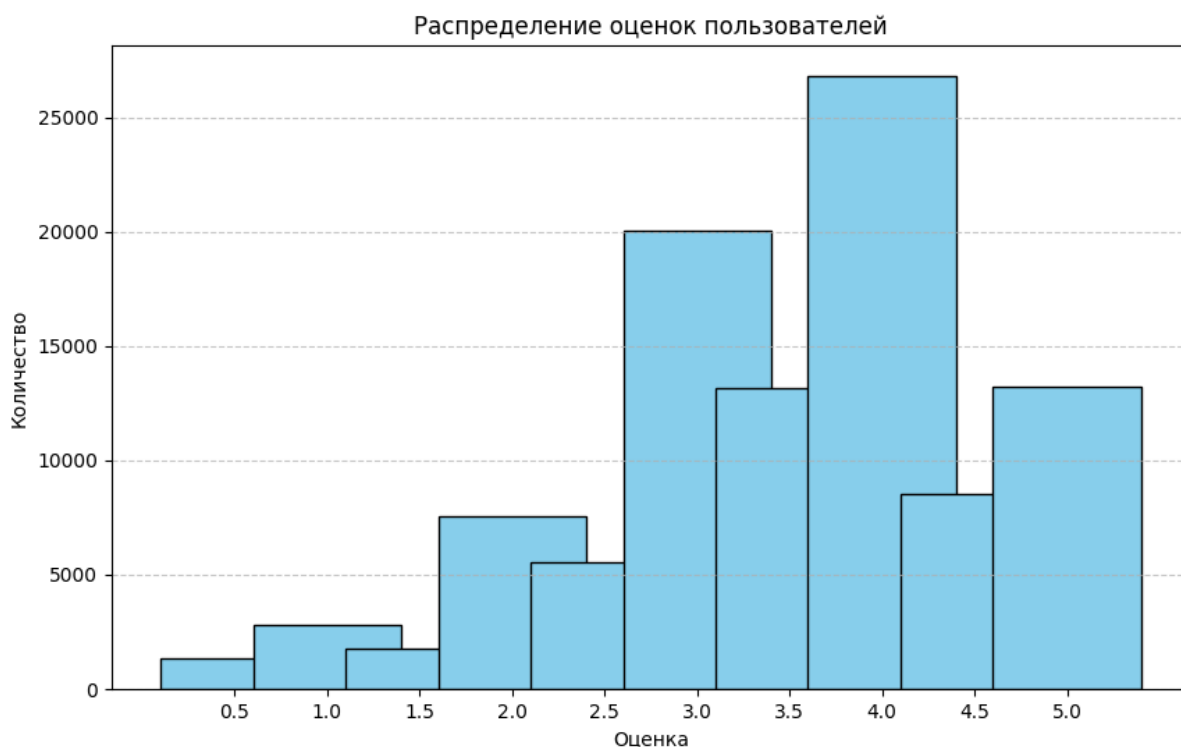
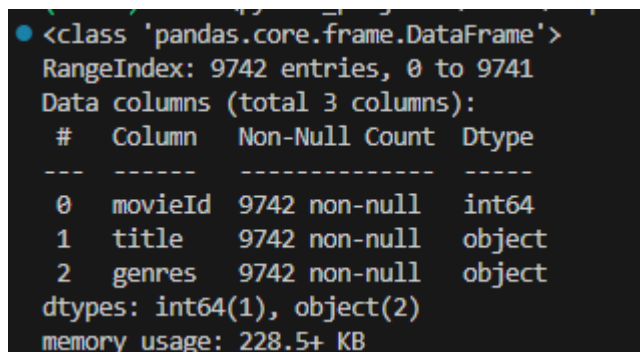


Рисунок 3.2.2 – Распределение оценок

Оценки распределены по шкале от 0.5 до 5 с шагом 0.5, что позволяет достаточно точно моделировать предпочтения пользователей, одновременно упрощая обработку данных. Анализ распределения оценок показывает, что наиболее популярные значения находятся в диапазоне от 3 до 4 баллов, что отражает склонность пользователей давать средние и положительные оценки.

Более экстремальные оценки, например 0.5 или 5, встречаются реже, но именно они могут быть особенно информативны при построении рекомендательных моделей, так как ясно отражают сильные предпочтения или отторжение.

Фильмы в датасете сопровождаются информацией о жанрах. Основная информация о данных в файле movies.csv отображена на Рисунке 3.2.3.



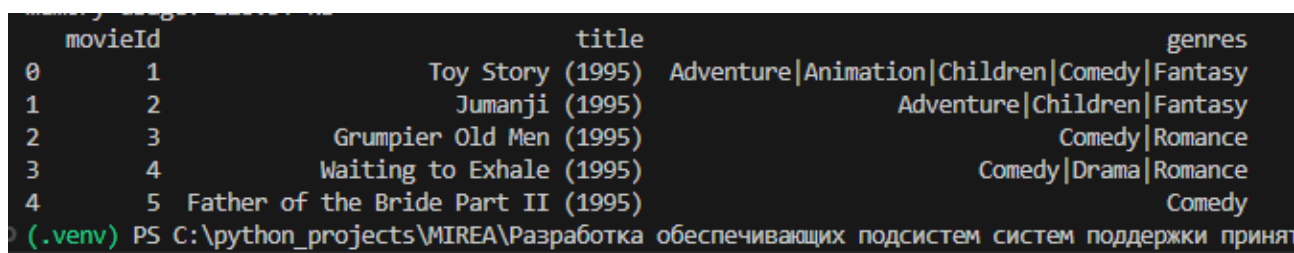
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9742 entries, 0 to 9741
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   movieId     9742 non-null   int64
1   title       9742 non-null   object
2   genres      9742 non-null   object
dtypes: int64(1), object(2)
memory usage: 228.5+ KB

```

**Рисунок 3.2.3 - Основная информация о данных в файле movies.csv**

Датафрейм не содержит нулевых значений или пропусков. Первые 5 строк датафрейма представлены на Рисунке 3.2.4.



|   | movieId | title                              | genres                                      |
|---|---------|------------------------------------|---------------------------------------------|
| 0 | 1       | Toy Story (1995)                   | Adventure Animation Children Comedy Fantasy |
| 1 | 2       | Jumanji (1995)                     | Adventure Children Fantasy                  |
| 2 | 3       | Grumpier Old Men (1995)            | Comedy Romance                              |
| 3 | 4       | Waiting to Exhale (1995)           | Comedy Drama Romance                        |
| 4 | 5       | Father of the Bride Part II (1995) | Comedy                                      |

**Рисунок 3.2.4 – Первые пять строк датафрейма movies**

Каждый фильм может относиться к нескольким жанрам, которые объединены в одну строку через символ «|». Анализ жанров показывает, что большинство фильмов имеют два-три жанра, что отражает реальную практику киноиндустрии, где произведения редко ограничиваются одним жанром. Также есть небольшое количество фильмов без указанных жанров, что требует дополнительной предобработки для корректного включения таких данных в рекомендательную систему.

В названиях фильмов часто содержится год выпуска, заключённый в скобки. Это позволяет извлечь дополнительную информацию о времени выхода фильма и анализировать данные по временным периодам.

Теги, оставленные пользователями, представляют собой текстовые описания, которые дают дополнительные сведения о предпочтениях. Информация о данных в файле tags.csv представлена на Рисунке 3.2.5.

```
><class 'pandas.core.frame.DataFrame'>
RangeIndex: 3683 entries, 0 to 3682
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   userId      3683 non-null   int64
1   movieId     3683 non-null   int64
2   tag         3683 non-null   object
3   timestamp   3683 non-null   int64
dtypes: int64(3), object(1)
memory usage: 115.2+ KB
```

Рисунок 3.2.5 - Основная информация о данных в файле tags.csv

Анализ тегов показывает, что пользователи используют их довольно разнообразно: одни ограничиваются жанровыми метками, другие оставляют более субъективные описания вроде «funny» или «thriller». Теги помогают расширить рекомендации, особенно в гибридных системах, где учитываются как оценки, так и семантические характеристики фильмов.

Временные метки, присутствующие как в рейтингах, так и в тегах, позволяют анализировать динамику активности пользователей. С их помощью можно определить, когда пользователи наиболее активно выставляли оценки, выявить сезонные колебания интереса к фильмам или построить модели, учитывающие эволюцию предпочтений во времени.

Общий анализ показывает, что датасет ml-latest-small хорошо сбалансирован для экспериментов с рекомендательными системами. Он содержит достаточно данных для выявления закономерностей в поведении пользователей и предпочтениях фильмов, но при этом не требует огромных вычислительных ресурсов, что позволяет быстро проводить предобработку и обучение моделей.

### 3.3 Предобработка данных

В рамках предобработки удален год выпуска фильма из названия и добавлен отдельный столбец `year` в соответствующий датафрейм.

Временные метки приведены к типу данных `datetime` для более удобной обработки. Жанры преобразованы в список, обработаны фильмы с отсутствующими жанрами.



## 4 ПРАКТИЧЕСКАЯ ЧАСТЬ

### 4.1 Функциональные возможности

В практической части реализована интерактивная консольная система для работы с данными MovieLens. Основная цель заключалась в создании среды, позволяющей пользователю просматривать фильмы, искать их по названию и жанрам, оценивать и получать рекомендации на основе собственных оценок и оценок других пользователей. Полный код реализации кинотеатра с рекомендательной системой представлен в Приложении А.

Главное меню кинотеатра представлено на Рисунке 4.1.1.

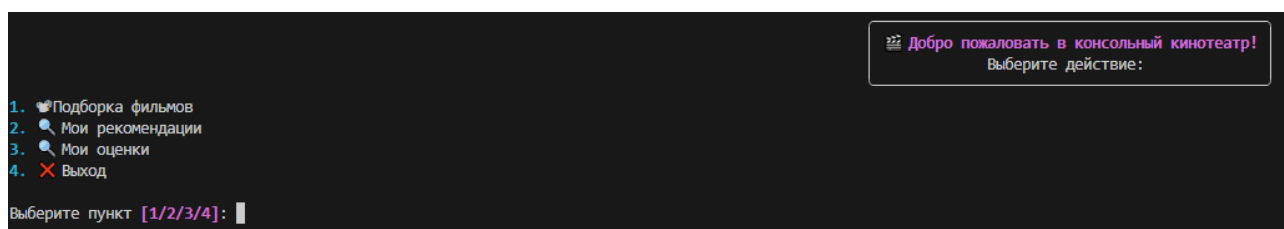


Рисунок 4.1.1 – Главное меню

Пользователю доступен просмотр каталога фильмов, ленты рекомендаций, ранее поставленных оценок, также реализована возможность выхода из приложения.

Каталог фильмов представлен на Рисунке 4.1.2.

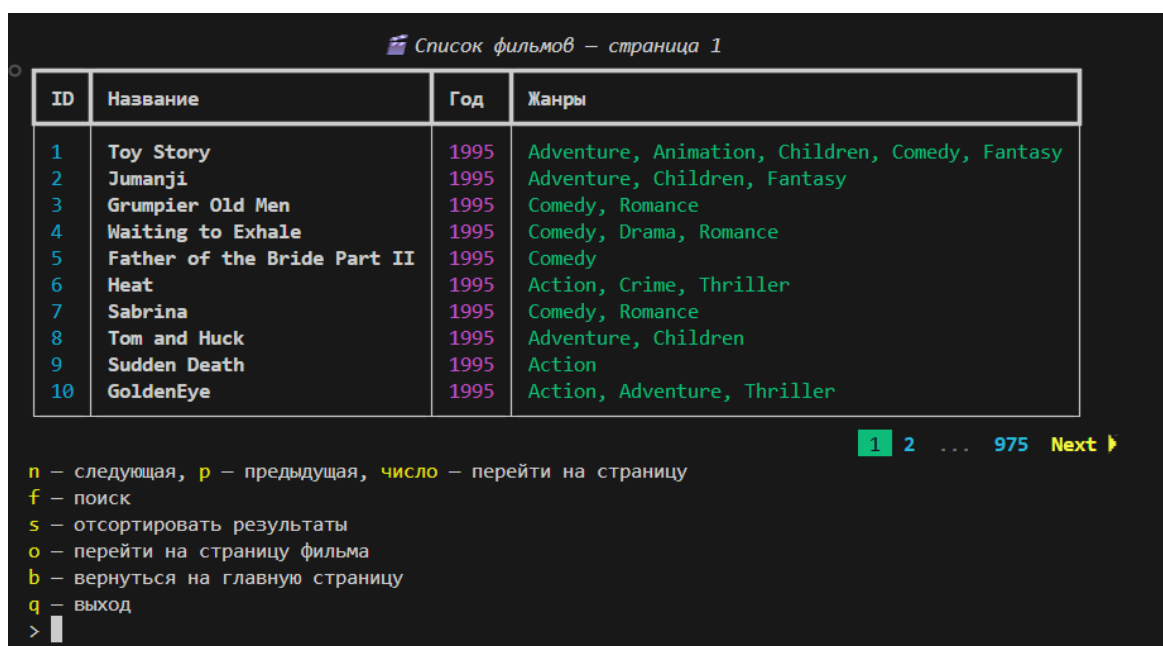


Рисунок 4.1.2 – Каталог фильмов

Сортировка каталога возможна по году выпуска фильм, по жанрам, по идентификатору. Пример сортировки каталога представлен на Рисунке 4.1.3.

Сортировка по Год (desc) – страница 1

| ID     | Название                    | Год  | Жанры                             |
|--------|-----------------------------|------|-----------------------------------|
| 183635 | Maze Runner: The Death Cure | 2018 | Action, Mystery, Sci-Fi, Thriller |
| 183959 | Tom Segura: Disgraceful     | 2018 | Comedy                            |
| 185033 | I Kill Giants               | 2018 | Drama, Fantasy, Thriller          |
| 188797 | Tag                         | 2018 | Comedy                            |
| 185473 | Blockers                    | 2018 | Comedy                            |
| 185435 | Game Over, Man!             | 2018 | Action, Comedy                    |
| 183611 | Game Night                  | 2018 | Action, Comedy, Crime, Horror     |
| 185585 | Pacific Rim: Uprising       | 2018 | Action, Fantasy, Sci-Fi           |
| 184471 | Tomb Raider                 | 2018 | Action, Adventure, Fantasy        |
| 183295 | Insidious: The Last Key     | 2018 | Horror, Mystery, Thriller         |

1 2 ... 975 Next ▶

n – следующая, p – предыдущая, число – перейти на страницу  
 o – открыть страницу фильма, b – назад  
 >

Рисунок 4.1.3 – Сортировка каталога по году по убыванию

Добавлен механизм поиска по каталогу с фильтрацией по году выпуска, жанрам. Поиск осуществляется по названию путем проверки заданной подстроки. Интерфейс поиска представлен на Рисунке 4.1.4.

Введите название фильма (Enter – пропустить): alien

Выберите жанры

| №  | Жанр        |
|----|-------------|
| 1  | Action      |
| 2  | Adventure   |
| 3  | Animation   |
| 4  | Children's  |
| 5  | Comedy      |
| 6  | Crime       |
| 7  | Documentary |
| 8  | Drama       |
| 9  | Fantasy     |
| 10 | Film-Noir   |
| 11 | Horror      |
| 12 | Musical     |
| 13 | Mystery     |
| 14 | Romance     |
| 15 | Sci-Fi      |
| 16 | Thriller    |
| 17 | War         |
| 18 | Western     |

Введите номера жанров через пробел (Enter – пропустить):

Рисунок 4.1.4 – Интерфейс поиска

Результат поиска на запрос alien представлены на Рисунке 4.1.5.

Результаты поиска: 20

| ID   | Название                                                                | Год  | Жанры                             |
|------|-------------------------------------------------------------------------|------|-----------------------------------|
| 1200 | Aliens                                                                  | 1986 | Action, Adventure, Horror, Sci-Fi |
| 1214 | Alien                                                                   | 1979 | Horror, Sci-Fi                    |
| 1320 | Alien³ (a.k.a. Alien 3)                                                 | 1992 | Action, Horror, Sci-Fi, Thriller  |
| 1690 | Alien: Resurrection                                                     | 1997 | Action, Horror, Sci-Fi            |
| 3701 | Alien Nation                                                            | 1988 | Crime, Drama, Sci-Fi, Thriller    |
| 4526 | My Stepmother Is an Alien                                               | 1988 | Comedy, Romance, Sci-Fi           |
| 5051 | Italian for Beginners (Italiensk for begyndere)                         | 2000 | Comedy, Drama, Romance            |
| 5704 | Without Warning (a.k.a. Alien Warning) (a.k.a. It Came Without Warning) | 1980 | Horror, Sci-Fi                    |
| 6835 | Alien Contamination                                                     | 1980 | Action, Horror, Sci-Fi            |
| 6899 | Alien from L.A.                                                         | 1988 | Sci-Fi                            |

1 2 Next ▶

Фильтры: Название: alien, Жанры: —, Год: —  
n — следующая, p — предыдущая, число — перейти на страницу  
s — отсортировать результаты  
o — открыть страницу фильма по ID  
r — сброс фильтров  
b — назад  
> |

Рисунок 4.1.5 – Результаты поиска на запрос alien

Для составлений рекомендаций пользователю необходимо поставить оценки фильмам. Для этого реализована возможность открытия «страницы» фильма, где доступен просмотр тэгов фильма и присутствует возможность оценить фильм.

Интерфейс страницы фильма представлен на Рисунке 4.1.6.

ID 1200

🎬

Название: Aliens

📅

Год: 1986

🏷️

Жанры: ['Action', 'Adventure', 'Horror', 'Sci-Fi']

Теги

action, aliens, horror, sci-fi, space, space craft, SPACE TRAVEL, suspense, space

Доступные действия:

r — оценить фильм  
b — назад  
q — выйти из приложения  
> |

Рисунок 4.1.6 – Интерфейс страницы фильма

Пример выставления оценки показан на Рисунке 4.1.7.

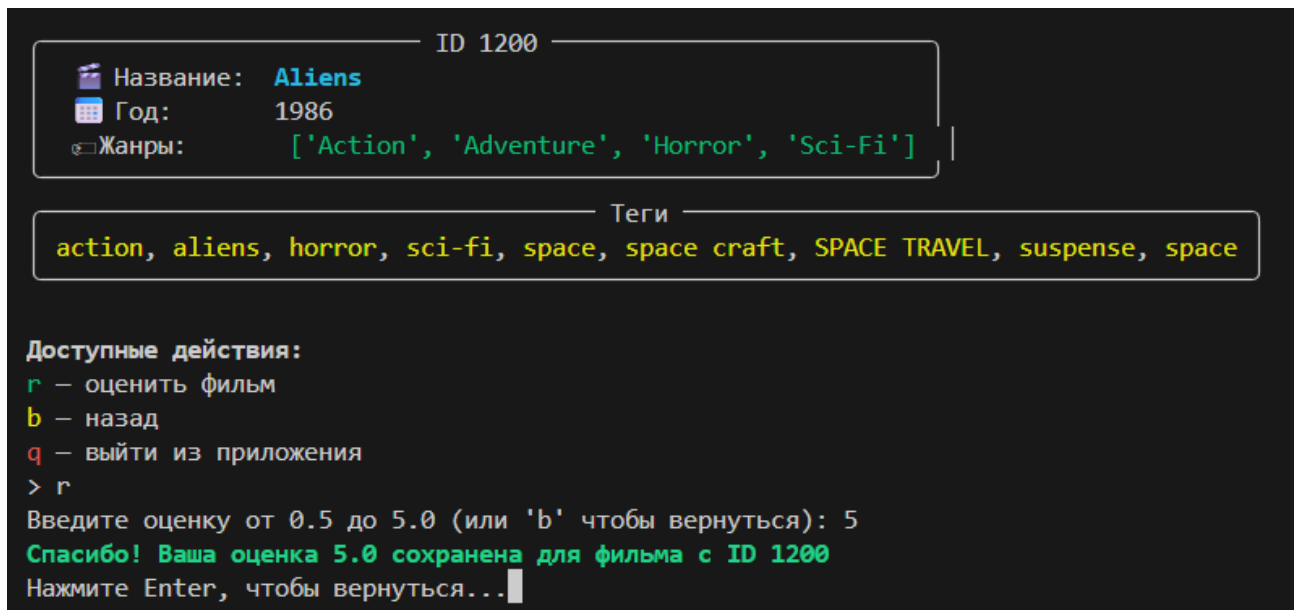


Рисунок 4.1.7 – Выставление оценки для фильма

Проставленные оценки можно посмотреть на отдельной странице в главном меню (Рисунок 4.1.8).

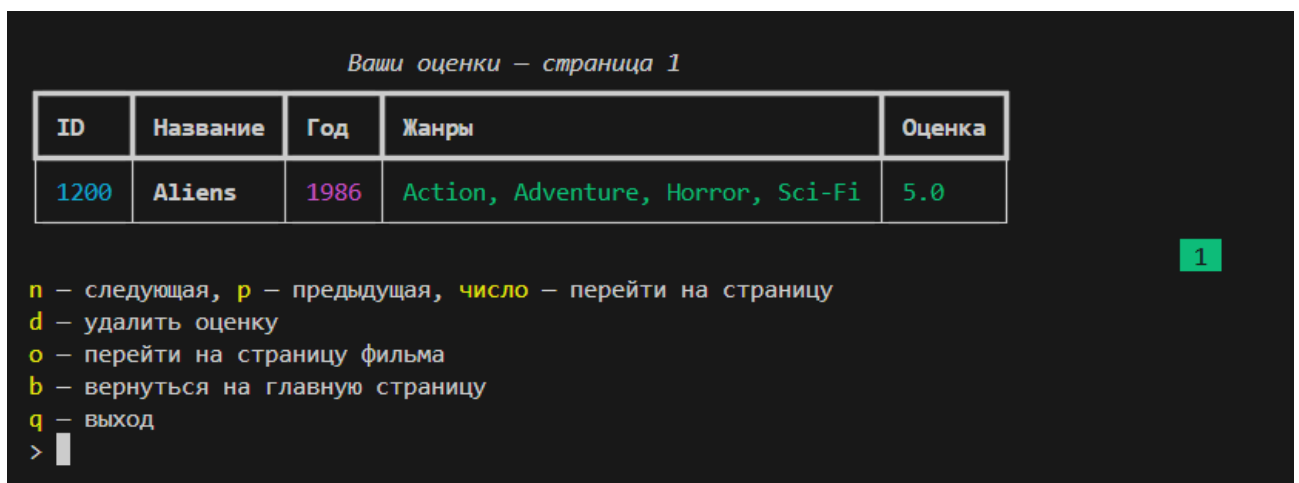


Рисунок 4.1.8 – Проставленные оценки

Добавлена возможность удалить оценку по идентификатору фильма (Рисунок 4.1.9).

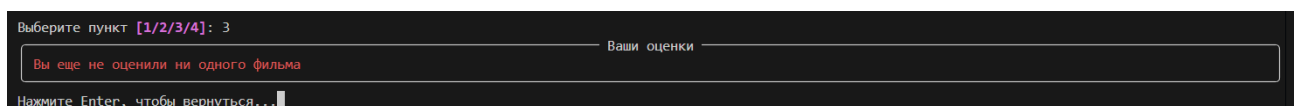


Рисунок 4.1.9 – Удаление оценки

Оценки на фильм с идентификатором 1200 успешно удалена, текущий пользователь не оценил ни одного фильма.

## 4.2 Сингулярное разложение

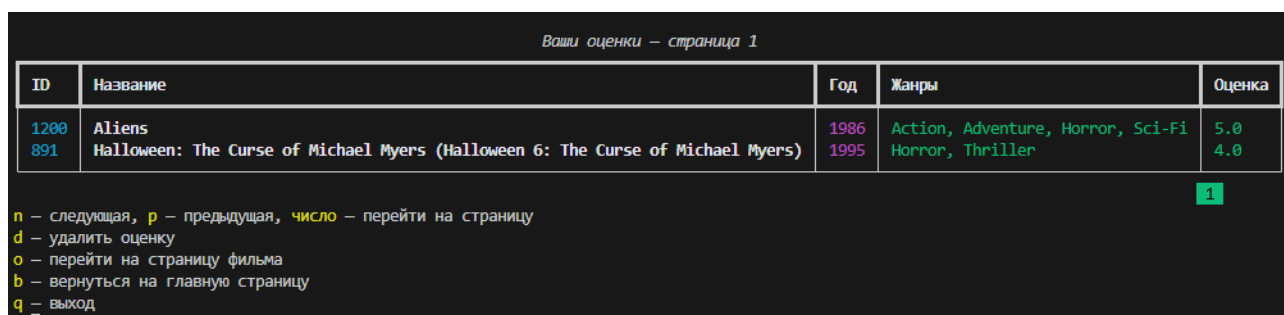
Для реализации системы рекомендаций на основе сингулярного разложения матрицы оценок написан класс `SVDRecommender`, представленный в файле `SVDRecommender.py` (Содержание файла представлено в Приложении Г).

Класс включает следующие методы:

- `init` — инициализация модели с указанием числа скрытых факторов  $k$ ;
- `build` — выполнение сингулярного разложения матрицы рейтингов и построение предсказанной матрицы;
- `evaluate` — вычисление метрик качества рекомендаций на реальных оценках;
- `recommend_for_current_user` — генерация персональных рекомендаций для текущего пользователя;
- `evaluate_over_k` — анализ зависимости качества модели от числа скрытых факторов с построением графика;
- `cluster_users` — кластеризация пользователей в пространстве скрытых признаков с визуализацией.

Для проверки рекомендаций задано 40 скрытых факторов при сингулярном разложении матрицы оценок.

Предположим, что пользователь выставил оценки, представленные на Рисунке 4.2.1.



| ID   | Название                                                                        | Год  | Жанры                             | Оценка |
|------|---------------------------------------------------------------------------------|------|-----------------------------------|--------|
| 1200 | Aliens                                                                          | 1986 | Action, Adventure, Horror, Sci-Fi | 5.0    |
| 891  | Halloween: The Curse of Michael Myers (Halloween 6: The Curse of Michael Myers) | 1995 | Horror, Thriller                  | 4.0    |

n – следующая, p – предыдущая, число – перейти на страницу  
d – удалить оценку  
o – перейти на страницу фильма  
b – вернуться на главную страницу  
q – выход

Рисунок 4.2.1 – Выставленные оценки

Полученные рекомендации представлены на Рисунке 4.2.2.

| SVD рекомендации – страница 1 |                                                                    |      |                                             |
|-------------------------------|--------------------------------------------------------------------|------|---------------------------------------------|
| ID                            | Название                                                           | Год  | Жанры                                       |
| 1214                          | Alien                                                              | 1979 | Horror, Sci-Fi                              |
| 1240                          | Terminator, The                                                    | 1984 | Action, Sci-Fi, Thriller                    |
| 589                           | Terminator 2: Judgment Day                                         | 1991 | Action, Sci-Fi                              |
| 1196                          | Star Wars: Episode V - The Empire Strikes Back                     | 1980 | Action, Adventure, Sci-Fi                   |
| 1387                          | Jaws                                                               | 1975 | Action, Horror                              |
| 541                           | Blade Runner                                                       | 1982 | Action, Sci-Fi, Thriller                    |
| 1197                          | Princess Bride, The                                                | 1987 | Action, Adventure, Comedy, Fantasy, Romance |
| 1201                          | Good, the Bad and the Ugly, The (Buono, il brutto, il cattivo, Il) | 1966 | Action, Adventure, Western                  |
| 1036                          | Die Hard                                                           | 1988 | Action, Crime, Thriller                     |
| 1210                          | Star Wars: Episode VI - Return of the Jedi                         | 1983 | Action, Adventure, Sci-Fi                   |

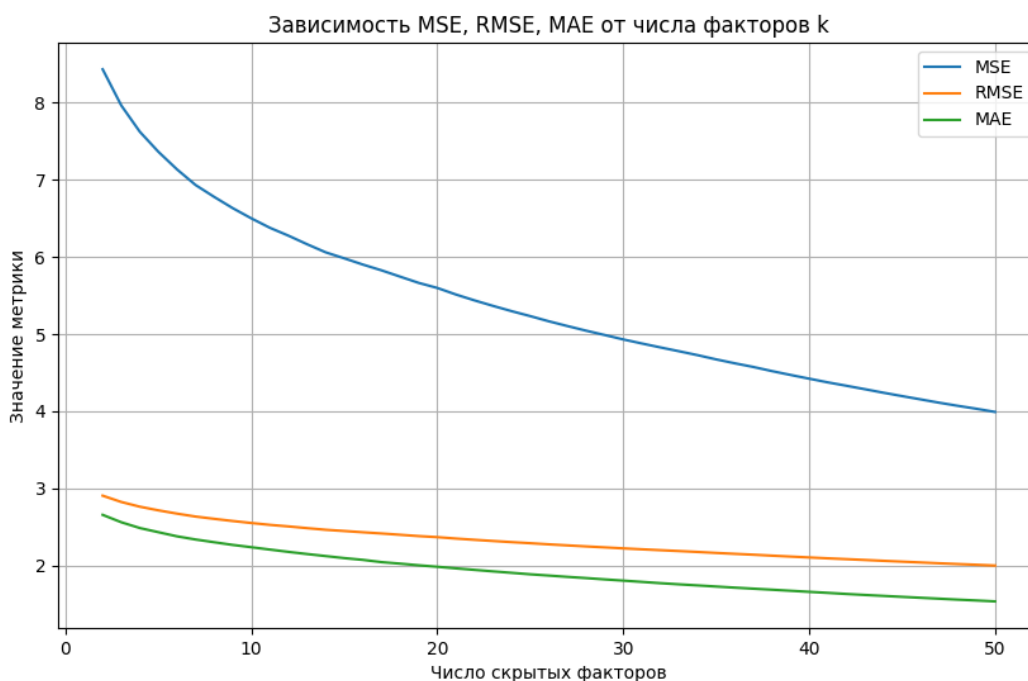
n – следующая, p – предыдущая, число – перейти на страницу  
 o – открыть страницу фильма, b – назад  
 >

**Рисунок 4.2.2 – Полученные рекомендации SVD**

На первом месте пользователю предлагается фильм «Alien» из серии фильмов «Чужой», что обусловлено тем, что текущий пользователь поставил оценку «5» другому фильму из этой вселенной — «Aliens». Это демонстрирует способность SVD выявлять семантические связи между фильмами, даже когда они имеют разные названия, но принадлежат к одной франшизе.

Большинство рекомендуемых фильмов относятся к жанрам «Action», «Sci-Fi» и «Horror», так как сингулярное разложение выявило, что данные жанры доминируют в профиле предпочтений пользователя на основе предоставленных им оценок.

Для определения оптимального числа скрытых факторов в сингулярном разложении построен график зависимости качества рекомендаций от числа скрытых факторов (Рисунок 4.2.3).



**Рисунок 4.2.3 - График зависимости качества рекомендаций от числа скрытых факторов**

По графику видно, что наиболее существенное улучшение метрик наблюдается при увеличении числа факторов от 2 до 20. В диапазоне 20–40 факторов наблюдается плавное снижение ошибок, а при  $k > 40$  улучшение становится незначительным, что свидетельствует о достижении точки насыщения. Это соответствует теоретическим ожиданиям, поскольку основные закономерности в данных захватываются первыми несколькими десятками скрытых факторов.

Дополнительно выводятся метрики качества рекомендаций для каждой построенной модели на основе сингулярного разложения при заданном числе скрытых факторов (Рисунок 4.2.4).

```

● Анализ влияния числа скрытых факторов SVD
○ Минимальное число скрытых факторов (2): 2
Максимальное число скрытых факторов (50): 50
Построение SVD для k=2: MSE: 8.4352, RMSE: 2.9043, MAE: 2.6556
Построение SVD для k=3: MSE: 7.9667, RMSE: 2.8225, MAE: 2.5596
Построение SVD для k=4: MSE: 7.6247, RMSE: 2.7613, MAE: 2.4845
Построение SVD для k=5: MSE: 7.3646, RMSE: 2.7138, MAE: 2.4304
Построение SVD для k=6: MSE: 7.1366, RMSE: 2.6714, MAE: 2.3768
Построение SVD для k=7: MSE: 6.9337, RMSE: 2.6332, MAE: 2.3357
Построение SVD для k=8: MSE: 6.7798, RMSE: 2.6038, MAE: 2.3003
Построение SVD для k=9: MSE: 6.6334, RMSE: 2.5755, MAE: 2.2654
Построение SVD для k=10: MSE: 6.5012, RMSE: 2.5497, MAE: 2.2355
Построение SVD для k=11: MSE: 6.3787, RMSE: 2.5256, MAE: 2.2051
Построение SVD для k=12: MSE: 6.2763, RMSE: 2.5052, MAE: 2.1750
Построение SVD для k=13: MSE: 6.1654, RMSE: 2.4830, MAE: 2.1472
Построение SVD для k=14: MSE: 6.0606, RMSE: 2.4618, MAE: 2.1206
Построение SVD для k=15: MSE: 5.9826, RMSE: 2.4459, MAE: 2.0943
Построение SVD для k=16: MSE: 5.9019, RMSE: 2.4294, MAE: 2.0714
Построение SVD для k=17: MSE: 5.8272, RMSE: 2.4140, MAE: 2.0422
Построение SVD для k=18: MSE: 5.7456, RMSE: 2.3970, MAE: 2.0226
Построение SVD для k=19: MSE: 5.6644, RMSE: 2.3800, MAE: 2.0011
Построение SVD для k=20: MSE: 5.5988, RMSE: 2.3662, MAE: 1.9820
Построение SVD для k=21: MSE: 5.5139, RMSE: 2.3482, MAE: 1.9614
Построение SVD для k=22: MSE: 5.4374, RMSE: 2.3318, MAE: 1.9416
Построение SVD для k=23: MSE: 5.3660, RMSE: 2.3165, MAE: 1.9222
Построение SVD для k=24: MSE: 5.2976, RMSE: 2.3017, MAE: 1.9035
Построение SVD для k=25: MSE: 5.2335, RMSE: 2.2877, MAE: 1.8844
Построение SVD для k=26: MSE: 5.1663, RMSE: 2.2729, MAE: 1.8680
Построение SVD для k=27: MSE: 5.1046, RMSE: 2.2593, MAE: 1.8511
Построение SVD для k=28: MSE: 5.0449, RMSE: 2.2461, MAE: 1.8353
Построение SVD для k=29: MSE: 4.9886, RMSE: 2.2335, MAE: 1.8184
Построение SVD для k=30: MSE: 4.9311, RMSE: 2.2206, MAE: 1.8024
Построение SVD для k=31: MSE: 4.8787, RMSE: 2.2088, MAE: 1.7860
Построение SVD для k=32: MSE: 4.8275, RMSE: 2.1972, MAE: 1.7693
Построение SVD для k=33: MSE: 4.7779, RMSE: 2.1859, MAE: 1.7543
Построение SVD для k=34: MSE: 4.7271, RMSE: 2.1742, MAE: 1.7403
Построение SVD для k=35: MSE: 4.6721, RMSE: 2.1615, MAE: 1.7257
Построение SVD для k=36: MSE: 4.6215, RMSE: 2.1498, MAE: 1.7113
Построение SVD для k=37: MSE: 4.5739, RMSE: 2.1387, MAE: 1.6978
Построение SVD для k=38: MSE: 4.5204, RMSE: 2.1261, MAE: 1.6846
Построение SVD для k=39: MSE: 4.4705, RMSE: 2.1144, MAE: 1.6707
Построение SVD для k=40: MSE: 4.4222, RMSE: 2.1029, MAE: 1.6570
Построение SVD для k=41: MSE: 4.3743, RMSE: 2.0915, MAE: 1.6435
Построение SVD для k=42: MSE: 4.3304, RMSE: 2.0810, MAE: 1.6293
Построение SVD для k=43: MSE: 4.2855, RMSE: 2.0701, MAE: 1.6164
Построение SVD для k=44: MSE: 4.2396, RMSE: 2.0590, MAE: 1.6043
Построение SVD для k=45: MSE: 4.1960, RMSE: 2.0484, MAE: 1.5918
Построение SVD для k=46: MSE: 4.1532, RMSE: 2.0379, MAE: 1.5798
Построение SVD для k=47: MSE: 4.1100, RMSE: 2.0273, MAE: 1.5680
Построение SVD для k=48: MSE: 4.0699, RMSE: 2.0174, MAE: 1.5556
Построение SVD для k=49: MSE: 4.0318, RMSE: 2.0079, MAE: 1.5448
Построение SVD для k=50: MSE: 3.9908, RMSE: 1.9977, MAE: 1.5332

```

**Рисунок 4.2.4 – Метрики качества для каждой построенной модели на основе сингулярного разложения при заданном числе скрытых факторов**

Из Рисунка 4.2.4 видно, что метрики MSE, RMSE, MAE монотонно уменьшаются с ростом числа скрытых факторов, что подтверждает способность модели более точно аппроксимировать исходную матрицу оценок при увеличении размерности пространства признаков.



## ЗАКЛЮЧЕНИЕ

В ходе выполнения данной практической работы успешно достигнута поставленная цель: разработана и реализована рекомендательная система, основанная на методах сингулярного разложения, позволяющая выявлять скрытые факторы пользовательских предпочтений.

Разработана функциональная консольная система на языке Python, интегрирующая интерактивный кинотеатр на основе датасета MovieLens и рекомендательный механизм на основе сингулярного разложения матрицы оценок. Система обеспечивает построение декомпозиции матрицы оценок, выделение скрытых факторов, генерацию персонализированных рекомендаций и кластеризацию пользователей в пространстве скрытых признаков.

Сингулярное разложение продемонстрировало высокую эффективность для выявления семантических связей между фильмами, что подтверждается качеством рекомендаций. Система успешно определяет тематические группы и франшизы, даже когда фильмы имеют различные названия, но принадлежат к одной вселенной.

Метрики качества рекомендаций показали монотонное улучшение с ростом числа скрытых факторов, что подтверждает способность модели точно аппроксимировать исходную матрицу оценок. Полученные значения ошибок находятся в приемлемом диапазоне для рекомендательных систем.

## СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1. Сорокин, А. Б. Безусловная оптимизация. [Электронный ресурс] : учебно-метод. пособие / А. Б. Сорокин, О. В. Платонова, Л. М. Железняк — М. РТУ МИРЭА , 2020.
2. Сорокин, А. Б. Введение в генетические алгоритмы: теория, расчеты и приложения. [Электронный ресурс] : учебно-метод. пособие / А. Б. Сорокин — М. МИРЭА , 2018.
3. Рекомендательные системы: user-based и item-based [Электронный ресурс]: Habr. URL: <https://habr.com/ru/companies/surfingbird/articles/139518/> (Дата обращения: 24.10.2025).
4. Рекомендательные системы [Электронный ресурс]: URL: [https://neerc.ifmo.ru/wiki/index.php?title=Рекомендательные\\_системы](https://neerc.ifmo.ru/wiki/index.php?title=Рекомендательные_системы) (Дата обращения: 22.10.2025).
5. MovieLens [Электронный ресурс]: URL: <https://grouplens.org/datasets/movielens/> (Дата обращения: 13.10.2025).
6. Сингулярное разложение — это очень полезно! [Электронный ресурс]: URL: <https://kpfu.ru/portal/docs/F984109526/SVD.pdf> (Дата обращения: 29.10.2025).
7. Сингулярное разложение матрицы [Электронный ресурс]: URL: <https://vmath.ru/vf5/algebra2/svd> (Дата обращения: 12.11.2025).

## ПРИЛОЖЕНИЯ

Приложение А — Реализация кинотеатра с встроенной рекомендательной системой, основанной на поиске скрытых факторов методом Марковских цепей.

Приложение Б — Код файла MarkovChain.py.

Приложение В — Код файла ContinuousMarkovChain.py.

Приложение Г — Код файла SVDR recommender.py.

## Приложение А

### Реализация кинотеатра с встроенной рекомендательной системой.

#### *Листинг А – Реализация кинотеатра с встроенной рекомендательной системой*

```
import os
import re
import pandas as pd
import numpy as np
from rich.console import Console
from rich.table import Table
from rich.prompt import Confirm, Prompt
from rich.panel import Panel
from rich.align import Align
from rich.text import Text
from enum import Enum
import seaborn as sns
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

from MarkovChain import MarkovChainRecommender
from ContinuousMarkovChain import ContinuousMarkovRecommender
from SVDRecommender import SVDRecommender

def extract_year(title):
    title = title.strip()
    match = re.search(r"\\(\\d{4})\\)\\s*$", title)
    if match:
        return int(match.group(1))
    return np.nan

class Genres(Enum):
    Action = "Action"
    Adventure = "Adventure"
    Animation = "Animation"
    Children = "Children's"
    Comedy = "Comedy"
    Crime = "Crime"
    Documentary = "Documentary"
    Drama = "Drama"
    Fantasy = "Fantasy"
    FilmNoir = "Film-Noir"
    Horror = "Horror"
    Musical = "Musical"
    Mystery = "Mystery"
    Romance = "Romance"
    SciFi = "Sci-Fi"
    Thriller = "Thriller"
    War = "War"
    Western = "Western"

class MovieLensCinema:
    def __init__(self, path, per_page=10, enable_markov_visualization=True):
        self.path = path
        self.links = None
        self.movies = None
        self.ratings = None
        self.tags = None
        self.console = Console()
        self.rates = pd.DataFrame(columns=["movieId",
"rating"]).set_index("movieId")
```

```

self.load_data()

self.per_page = per_page
self.markov_recommender = MarkovChainRecommender(
    self, auto_visualize=enable_markov_visualization
)
self.continuous_markov_recommender = ContinuousMarkovRecommender(
    self, auto_visualize=enable_markov_visualization
)
self.svd = SVDRecommender(self, k=40)

def load_data(self):
    links_path = os.path.join(self.path, "links.csv")
    movies_path = os.path.join(self.path, "movies.csv")
    ratings_path = os.path.join(self.path, "ratings.csv")
    tags_path = os.path.join(self.path, "tags.csv")

    if os.path.exists(links_path):
        self.links = pd.read_csv(
            links_path,
            encoding="utf-8",
            index_col="movieId",
            dtype={"imdbId": "int64", "tmdbId": "Int64"},
        )
    else:
        raise FileNotFoundError("Не найдено файла links.csv")

    if os.path.exists(movies_path):
        self.movies = pd.read_csv(
            movies_path, encoding="utf-8", index_col="movieId",
quotechar="'"
        )
        self.movies["title"] = self.movies["title"].str.strip()
        self.movies["genres"] = self.movies["genres"].apply(
            lambda x: [] if x == "(no genres listed)" else x.split("|")
        )
        self.movies["year"] = self.movies["title"].apply(extract_year)
        self.movies["year"] = self.movies["year"].astype("Int64")
        self.movies["title"] = self.movies["title"].apply(
            lambda t: re.sub(r"\s*(\d{4})$", "", t)
        )
    else:
        raise FileNotFoundError("Не найдено файла movies.csv")

    if os.path.exists(ratings_path):
        self.ratings = pd.read_csv(
            ratings_path,
            encoding="utf-8",
            dtype={
                "userId": "int32",
                "movieId": "int32",
                "rating": "float32",
                "timestamp": "int64",
            },
        )
        self.ratings["datetime"] = pd.to_datetime(
            self.ratings["timestamp"], unit="s"
        )
        self.ratings = self.ratings.drop(columns=["timestamp"])
        self.ratings.set_index(["userId", "movieId"], inplace=True)

```

```

else:
    raise FileNotFoundError("Не найдено файла ratings.csv")

if os.path.exists(tags_path):
    self.tags = pd.read_csv(
        tags_path,
        dtype={
            "userId": "int32",
            "movieId": "int32",
            "tag": "string",
            "timestamp": "int64",
        },
        quotechar='"',
    )
    self.tags["datetime"] = pd.to_datetime(self.tags["timestamp"],
unit="s")
    self.tags.set_index(["userId", "movieId"], inplace=True)
else:
    raise FileNotFoundError("Не найдено файла tags.csv")

def get_rating_matrix(self):
    df = self.ratings.reset_index()

    rating_matrix = df.pivot_table(
        index="userId", columns="movieId", values="rating"
    )

    return rating_matrix

def show_movies(self, page=1):
    start = (page - 1) * self.per_page
    end = start + self.per_page
    subset = self.movies.iloc[start:end]

    table = Table(title=f"🎬 Список фильмов — страница {page}")

    table.add_column("ID", style="cyan", no_wrap=True)
    table.add_column("Название", style="bold")
    table.add_column("Год", justify="center", style="magenta")
    table.add_column("Жанры", style="green")

    for movie_id, row in subset.iterrows():
        genres_str = ", ".join(row["genres"]) if row["genres"] else "-"
        year_str = str(row["year"]) if not pd.isna(row["year"]) else "-"
        table.add_row(str(movie_id), row["title"], year_str, genres_str)

    self.console.print(table)

def run(self):
    page = 1
    total_pages = (len(self.movies) - 1) // self.per_page + 1

    while True:
        os.system("cls" if os.name == "nt" else "clear")
        self.show_movies(page=page)

        self.show_paginator(page, total_pages)

        self.console.print(
            "[yellow]n[/yellow] — следующая, [yellow]p[/yellow] —
предыдущая, "

```

```

        "[yellow]число[/yellow] – перейти на страницу\n"
        "[yellow]f[/yellow] – поиск\n"
        "[yellow]s[/yellow] – отсортировать результаты\n"
        "[yellow]o[/yellow] – перейти на страницу фильма\n"
        "[yellow]b[/yellow] – вернуться на главную страницу\n"
        "[yellow]q[/yellow] – выход"
    )

    choice = input("> ").strip().lower()
    if choice == "n" and page < total_pages:
        page += 1
    elif choice == "p" and page > 1:
        page -= 1
    elif choice.isdigit():
        num = int(choice)
        if 1 <= num <= total_pages:
            page = num
    elif choice == "f":
        self.search_movies()
    elif choice == "s":
        self.sort_movies(self.movies)
    elif choice == "o":
        movie_id_str = Prompt.ask("Введите ID фильма (или 'b' для
отмены) ")
        if movie_id_str.lower() == "b":
            continue
        if movie_id_str.isdigit():
            self.show_movie_page(int(movie_id_str))
        else:
            self.console.print("[red]Некорректный ID[/red]")
            input("Нажмите Enter, чтобы вернуться...")
    elif choice == "b":
        break
    elif choice == "q":
        if Confirm.ask("Are you sure?"):
            exit(0)

    def menu(self):
        while True:
            os.system("cls" if os.name == "nt" else "clear")

            self.console.print(
                Panel.fit(
                    "[bold magenta]🎬 Добро пожаловать в консольный
кинотеатр![/bold magenta]\nВыберите действие:"
                ),
                justify="center",
            )

            self.console.print("[cyan]1.[/cyan] 🎬 Подборка фильмов")
            self.console.print("[cyan]2.[/cyan] 🔍 Мои рекомендации")
            self.console.print("[cyan]3.[/cyan] 🗣️ Мои оценки")
            self.console.print("[cyan]4.[/cyan] 📊 График зависимости качества
рекомендаций от числа скрытых факторов (SVD)")
            self.console.print("[cyan]5.[/cyan] 🧠 Кластеризация пользователей
(SVD) ")
            self.console.print("[cyan]6.[/cyan] ❌ Выход")

            choice = Prompt.ask("\nВыберите пункт", choices=["1", "2", "3", "4",
"5"])

```

```

        if choice == "1":
            self.run()
        elif choice == "2":
            self.show_recommendations()
        elif choice == "3":
            self.show_my_ratings()
        elif choice == "4":
            self.svd_analyze()
        elif choice == "5":
            self.cluster_menu()
        elif choice == "6":
            if Confirm.ask("[bold red]Вы действительно хотите выйти?[/bold
red]"):
                break

    def show_movie_page(self, movie_id: int):
        """Страница фильма с информацией, тегами и возможностью оценки"""
        if movie_id not in self.movies.index:
            self.console.print(f"[red]Фильм с ID {movie_id} не найден.[/red]")
            input("Нажмите Enter, чтобы вернуться...")
            return

        while True:
            os.system("cls" if os.name == "nt" else "clear")

            movie = self.movies.loc[movie_id]

            info_table = Table(show_header=False, box=None)
            info_table.add_row(
                "🎬 Название:", f"[bold cyan]{movie['title']}[/bold cyan]"
            )
            info_table.add_row(
                "📅 Год:", str(movie["year"]) if movie["year"] == movie["year"]
            )
            info_table.add_row("🎭 Жанры:", f"[green]{movie['genres']}[/green]")
            self.console.print(Panel(info_table, title=f"ID {movie_id}",
            expand=False))

            if self.tags is not None:
                try:
                    df = self.tags.xs(movie_id, level="movieId")
                    movie_tags = df["tag"].tolist()
                except KeyError:
                    movie_tags = []

                if movie_tags:
                    tags_str = ", ".join(f"[yellow]{t}[/yellow]" for t in
movie_tags)
                    self.console.print(Panel(tags_str, title="Теги",
expand=False))
                else:
                    self.console.print(
                        Panel(
                            "[italic grey]Для этого фильма нет тегов[/italic
grey]",
                            title="Теги",
                            expand=False,
                        )
                    )
            )
    )

```



```

        self.console.print(
            "\n[bold]Доступные действия:[/bold]\n"
            "[green]r[/green] – оценить фильм\n"
            "[yellow]b[/yellow] – назад\n"
            "[red]q[/red] – выйти из приложения"
        )

        choice = input("> ").strip().lower()
        if choice == "r":
            self.rate_movie(movie_id)
        elif choice == "b":
            break
        elif choice == "q":
            if Confirm.ask("Are you sure?"):
                exit(0)

    def rate_movie(self, movie_id: int):
        """Простейшая система выставления оценки"""
        while True:
            rating = Prompt.ask(
                "Введите оценку от 0.5 до 5.0 (или 'b' чтобы вернуться)"
            )
            if rating.lower() == "b":
                break
            try:
                rating = float(rating)
                if 0.5 <= rating <= 5.0:
                    self.rates.loc[movie_id, "rating"] = rating
                    self.markov_recommender.reset_matrix()
                    self.continuous_markov_recommender.reset_matrix()
                    self.markov_recommender.build_transition_matrix()
                    self.continuous_markov_recommender.build_intensity_matrix()
                    self.console.print(
                        f"[bold green]Спасибо! Ваша оценка {rating} сохранена
для фильма с ID {movie_id}[/bold green]"
                    )
                    input("Нажмите Enter, чтобы вернуться...")
                    break
                else:
                    self.console.print(
                        "[red]Оценка должна быть в диапазоне 0.5–5.0[/red]"
                    )
                    input("Нажмите Enter, чтобы вернуться...")
                    break
            except ValueError:
                self.console.print("[red]Введите корректное число или
'b'[/red]")
                input("Нажмите Enter, чтобы вернуться...")
                break

    def search_movies(self):
        """Поиск фильмов по названию с фильтрацией по году и жанрам"""
        os.system("cls" if os.name == "nt" else "clear")
        search_query = (
            Prompt.ask("Введите название фильма (Enter –
пропустить)").strip().lower()
        )
        selected_genres = self.choose_genres()
        year_op, year_val = self.choose_year_filter()

```

```

def apply_filters():
    df = self.movies
    if search_query:
        df = df[df["title"].str.lower().str.contains(search_query,
na=False)]
    if selected_genres:
        df = df[
            df["genres"].apply(
                lambda g: all(gen in g for gen in selected_genres)
            )
        ]
    if year_op is not None and year_val is not None:
        if year_op == "=":
            df = df[df["year"] == year_val]
        elif year_op == ">":
            df = df[df["year"] > year_val]
        elif year_op == "<":
            df = df[df["year"] < year_val]
    return df

filtered_df = apply_filters()
if filtered_df.empty:
    self.console.print(
        Panel("[red]❌ Фильмы не найдены[/red]", title="Результат")
    )
    input("Нажмите Enter, чтобы вернуться...")
    return

page = 1
per_page = self.per_page

while True:
    os.system("cls" if os.name == "nt" else "clear")

    total_pages = max(1, (len(filtered_df) - 1) // per_page + 1)
    page = min(page, total_pages)

    start = (page - 1) * per_page
    end = start + per_page
    page_data = filtered_df.iloc[start:end]

    table = Table(
        title=f"Результаты поиска: {len(filtered_df)}",
        show_header=True,
        header_style="bold magenta",
    )
    table.add_column("ID", style="cyan", width=6)
    table.add_column("Название", style="white")
    table.add_column("Год", style="yellow", width=8)
    table.add_column("Жанры", style="green")

    for idx, row in page_data.iterrows():
        year = str(row["year"]) if not pd.isna(row["year"]) else "-"
        genres = ", ".join(row["genres"]) if row["genres"] else "-"
        table.add_row(str(idx), row["title"], year, genres)
    self.console.print(table)
    self.show_paginator(page, total_pages)
    status = f"[bold]Фильтры:[/bold] Название: [cyan]{search_query or '-'}[/cyan], Жанры: [cyan]{'', '.join(selected_genres) if selected_genres else '-'}[/cyan], Год: [cyan]{year_op + str(year_val) if year_op and year_val else '-'}[/cyan]"

```

```

        self.console.print(status)

        self.console.print(
            "[yellow]n[/yellow] – следующая, [yellow]p[/yellow] –
предыдущая, [yellow]число[/yellow] – перейти на страницу\n"
            "[yellow]s[/yellow] – отсортировать результаты\n"
            "[yellow]o[/yellow] – открыть страницу фильма по ID\n"
            "[yellow]r[/yellow] – сброс фильтров\n"
            "[yellow]b[/yellow] – назад"
        )

        choice = input("> ").strip().lower()
        if choice == "n" and page < total_pages:
            page += 1
        elif choice == "p" and page > 1:
            page -= 1
        elif choice == "s":
            self.sort_movies(filtered_df)
        elif choice.isdigit():
            num = int(choice)
            if 1 <= num <= total_pages:
                page = num
        elif choice == "o":
            movie_id_str = Prompt.ask("Введите ID фильма (или 'b' чтобы
вернуться)")
            if movie_id_str.lower() == "b":
                continue
            try:
                movie_id = int(movie_id_str)
                if movie_id in self.movies.index:
                    self.show_movie_page(movie_id)
            else:
                self.console.print("[red]Фильм с таким ID не
найден[/red]")
                input("Нажмите Enter, чтобы продолжить...")
        except ValueError:
            self.console.print("[red]Введите корректный ID[/red]")
            input("Нажмите Enter, чтобы продолжить...")
        elif choice == "r":
            search_query = ""
            selected_genres = []
            year_op, year_val = None, None
            filtered_df = self.movies
            page = 1
        elif choice == "b":
            break

    def choose_genres(self):
        """Выбор одного или нескольких жанров через консоль с таблицей"""
        genre_list = list(Genres)
        table = Table(
            title="Выберите жанры", show_header=True, header_style="bold
magenta"
        )
        table.add_column("№", justify="center", style="cyan", width=4)
        table.add_column("Жанр", justify="left", style="green")

        for i, genre in enumerate(genre_list, 1):
            table.add_row(str(i), genre.value)

        self.console.print(table)

```

### Продолжение Листинга А

```
choice = Prompt.ask(
    "Введите номера жанров через пробел (Enter — пропустить)"
).strip()
if not choice:
    return []

selected_genres = []
for num in choice.split():
    num = num.strip()
    if num.isdigit():
        idx = int(num) - 1
        if 0 <= idx < len(genre_list):
            selected_genres.append(genre_list[idx].value)
return selected_genres

def choose_year_filter(self):
    """Выбор фильтра по году с оператором"""
    op = Prompt.ask(
        "Выберите оператор для фильтра по году",
        choices=[">", "<", "="],
        default="=",
    )
    year_str = Prompt.ask("Введите год").strip()
    if not year_str.isdigit():
        self.console.print("[red]Некорректный год, фильтр не будет
применён[/red]")
        return None, None
    return op, int(year_str)

def show_paginator(self, page, total_pages):
    """Красивый центрированный пагинатор"""
    paginator_text = Text()
    last_was_ellipsis = False

    if page > 1:
        paginator_text.append("◀ Prev ", style="bold yellow")
    else:
        paginator_text.append(" ")

    for p in range(1, total_pages + 1):
        if p == 1 or p == total_pages or abs(p - page) <= 1:
            if p == page:
                paginator_text.append(f" {p} ", style="reverse green")
            else:
                paginator_text.append(f" {p} ", style="bold cyan")
            last_was_ellipsis = False
        else:
            if not last_was_ellipsis:
                paginator_text.append(" ... ", style="dim")
                last_was_ellipsis = True
    if page < total_pages:
        paginator_text.append(" Next ►", style="bold yellow")

    self.console.print(Align.center(paginator_text))

def show_my_ratings(self):
    if self.rates.empty:
        self.console.print(
            Panel(
                "[red]Вы еще не оценили ни одного фильма[/red]", title="Ваши
оценки"
```

```

    )
    input("Нажмите Enter, чтобы вернуться...")
    return

page = 1
df = self.rates.merge(self.movies, on="movieId")
total_pages = (len(df) - 1) // self.per_page + 1

while len(df):
    os.system("cls" if os.name == "nt" else "clear")

    start = (page - 1) * self.per_page
    end = start + self.per_page
    subset = df.iloc[start:end]

    rates = Table(show_header=True, title=f"Ваши оценки – страница
{page}")

    rates.add_column("ID", style="cyan", no_wrap=True)
    rates.add_column("Название", style="bold")
    rates.add_column("Год", justify="center", style="magenta")
    rates.add_column("Жанры", style="green")
    rates.add_column("Оценка", style="green")

    for movie_id, row in subset.iterrows():
        genres_str = ", ".join(row["genres"]) if row["genres"] else "-"
        year_str = str(row["year"]) if not pd.isna(row["year"]) else "-"
        rates.add_row(
            str(movie_id),
            row["title"],
            year_str,
            genres_str,
            str(row["rating"]),
        )

    self.console.print(rates)

    self.show_paginator(page, total_pages)

    self.console.print(
        "[yellow]n[/yellow] – следующая, [yellow]p[/yellow] –
предыдущая, "
        "[yellow]число[/yellow] – перейти на страницу\n"
        "[yellow]d[/yellow] – удалить оценку\n"
        "[yellow]o[/yellow] – перейти на страницу фильма\n"
        "[yellow]b[/yellow] – вернуться на главную страницу\n"
        "[yellow]q[/yellow] – выход"
    )
    choice = input("> ").strip().lower()
    if choice == "n" and page < total_pages:
        page += 1
    elif choice == "p" and page > 1:
        page -= 1
    elif choice.isdigit():
        num = int(choice)
        if 1 <= num <= total_pages:
            page = num
    elif choice == "d":
        movie_id_str = Prompt.ask(
            "Введите ID фильма, у которого хотите удалить оценку (или
'b' для отмены)"

```

```

        )
        if movie_id_str.lower() == "b":
            continue
        if movie_id_str.isdigit():
            id = int(movie_id_str)
            if id in self.rates.index:
                self.rates = self.rates.drop(index=id)
                df = df.drop(index=id)
                self.markov_recommender.reset_matrix()
                self.markov_recommender.build_transition_matrix()
            else:
                self.console.print(
                    "[red]Не найдено оценки для фильма с заданным
ID[/red]"
                )
                input("Нажмите Enter, чтобы вернуться...")
        else:
            self.console.print("[red]Некорректный ID[/red]")
            input("Нажмите Enter, чтобы вернуться...")
    elif choice == "o":
        movie_id_str = Prompt.ask("Введите ID фильма (или 'b' для
отмены)")
        if movie_id_str.lower() == "b":
            continue
        if movie_id_str.isdigit():
            self.show_movie_page(int(movie_id_str))
        else:
            self.console.print("[red]Некорректный ID[/red]")
            input("Нажмите Enter, чтобы вернуться...")
    elif choice == "b":
        break
    elif choice == "q":
        if Confirm.ask("Are you sure?"):
            exit(0)

def sort_movies(self, frame):
    """Сортировка фильмов по различным полям"""
    sort_fields = [
        ("ID", "movieId"),
        ("Название", "title"),
        ("Год", "year"),
        ("Жанры", "genres"),
    ]

    table = Table(
        title="Выберите поле для сортировки",
        show_header=True,
        header_style="bold magenta",
    )
    table.add_column("№", justify="center", style="cyan", width=4)
    table.add_column("Поле", style="green")

    for i, (label, _) in enumerate(sort_fields, 1):
        table.add_row(str(i), label)

    self.console.print(table)

    choice = Prompt.ask("Введите номер поля (или Enter для отмены)").strip()

    if not choice.isdigit():
        return

```

```

        choice_num = int(choice)
        if not 1 <= choice_num <= len(sort_fields):
            self.console.print("[red]Некорректный выбор[/red]")
            input("Нажмите Enter, чтобы вернуться...")
            return

        field_label, field_name = sort_fields[choice_num - 1]

        direction = Prompt.ask(
            "Выберите направление сортировки", choices=["asc", "desc"],
            default="asc"
        )

        ascending = direction == "asc"

        if field_name == "movieId":
            sorted_df = frame.sort_index(ascending=ascending)
        elif field_name == "genres":
            sorted_df = frame.copy()
            sorted_df["__sort_genre"] = sorted_df["genres"].apply(
                lambda g: g[0] if g else ""
            )
            sorted_df = sorted_df.sort_values("__sort_genre",
            ascending=ascending).drop(
                columns="__sort_genre"
            )
        else:
            sorted_df = frame.sort_values(field_name, ascending=ascending)

        self.paginated_view(
            sorted_df, title=f"Сортировка по {field_label} ({direction})"
        )

    def paginated_view(self, df, title="Список фильмов"):
        """Универсальный постраничный просмотр DataFrame фильмов"""
        page = 1
        total_pages = max(1, (len(df) - 1) // self.per_page + 1)

        while True:
            os.system("cls" if os.name == "nt" else "clear")
            start = (page - 1) * self.per_page
            end = start + self.per_page
            subset = df.iloc[start:end]

            table = Table(title=f"{title} - страница {page}", show_header=True)
            table.add_column("ID", style="cyan", no_wrap=True)
            table.add_column("Название", style="bold")
            table.add_column("Год", justify="center", style="magenta")
            table.add_column("Жанры", style="green")

            for movie_id, row in subset.iterrows():
                genres_str = ", ".join(row["genres"]) if row["genres"] else "-"
                year_str = str(row["year"]) if not pd.isna(row["year"]) else "-"
                table.add_row(str(movie_id), row["title"], year_str, genres_str)

            self.console.print(table)
            self.show_paginator(page, total_pages)
            self.console.print(
                "[yellow]n[/yellow] - следующая, [yellow]p[/yellow] -
предыдущая, "

```

### Продолжение Листинга А

```
        "[yellow]число[/yellow] – перейти на страницу\n"
        "[yellow]o[/yellow] – открыть страницу фильма,
[yellow]b[/yellow] – назад"
    )

    choice = input("> ").strip().lower()
    if choice == "n" and page < total_pages:
        page += 1
    elif choice == "p" and page > 1:
        page -= 1
    elif choice.isdigit():
        num = int(choice)
        if 1 <= num <= total_pages:
            page = num
    elif choice == "o":
        movie_id_str = Prompt.ask("Введите ID фильма (или 'b' для
отмены) ")
        if movie_id_str.lower() == "b":
            continue
        if movie_id_str.isdigit():
            self.show_movie_page(int(movie_id_str))
    elif choice == "b":
        break
    os.system("cls" if os.name == "nt" else "clear")

def show_recommendations(self):
    """Меню выбора стратегии рекомендаций"""
    os.system("cls" if os.name == "nt" else "clear")
    if self.rates.empty():
        self.console.print(
            Panel(
                "[red]У вас нет оценок для генерации рекомендаций.[/red]",
                title="Рекомендации",
            )
        )
    input("Нажмите Enter, чтобы вернуться...")
    return

    strategy = Prompt.ask(
        "Выберите стратегию рекомендаций",
        choices=["user-based", "item-based", "markov-discrete", "markov-
continuous", "svd"],
        default="user-based",
    )

    if strategy == "user-based":
        self.recommend_user_based()
    elif strategy == "item-based":
        self.recommend_item_based()
    elif strategy == "markov-discrete":
        self.recommend_markov_chain()
    elif strategy == "svd":
        self.recommend_svd()
    else:
        self.recommend_continuous_markov_chain()

def recommend_user_based(self):
    """User-based рекомендации"""
    R = self.get_rating_matrix()
    user_ids = R.index.tolist()
    movie_ids = R.columns.tolist()
```



### Продолжение Листинга А

```
current_user_id = -1

current_user_ratings = pd.Series(
    [np.nan] * len(movie_ids), index=movie_ids, dtype=float
)
for movie_id in self.rates.index:
    if movie_id in movie_ids:
        current_user_ratings[movie_id] = self.rates.loc[movie_id,
"rating"]

R = pd.concat(
    [R, pd.DataFrame([current_user_ratings], index=[current_user_id])]
)
current_user_idx = R.index.get_loc(current_user_id)

metric = Prompt.ask(
    "Выберите метрику сходства [pearson/jaccard/lp/otiai]",
    choices=["pearson", "jaccard", "lp", "otiai"],
    default="pearson",
)

R_values = R.values.astype(np.float64)
n_users, n_items = R_values.shape
user_means = np.nanmean(R_values, axis=1)
preds = np.full(n_items, np.nan)
sims = np.zeros(n_users)
current_ratings = R_values[current_user_idx, :]

for u in range(n_users):
    if u == current_user_idx:
        continue
    other = R_values[u, :]
    mask = ~np.isnan(current_ratings) & ~np.isnan(other)
    if np.sum(mask) == 0:
        continue
    if metric == "pearson":
        sims[u] = np.corrcoef(current_ratings[mask], other[mask])[0, 1]
    elif metric == "lp":
        sims[u] = -np.linalg.norm(current_ratings[mask] - other[mask])
    elif metric == "jaccard":
        sims[u] = np.sum(
            (current_ratings[mask] > 0) & (other[mask] > 0)
        ) / np.sum((current_ratings[mask] > 0) | (other[mask] > 0))
    elif metric == "otiai":
        sims[u] = np.dot(current_ratings[mask], other[mask]) / (
            np.linalg.norm(current_ratings[mask]) *
np.linalg.norm(other[mask])
        )

    for i in range(n_items):
        if not np.isnan(R_values[current_user_idx, i]):
            continue
        mask = ~np.isnan(R_values[:, i])
        if np.sum(mask) == 0:
            continue
        numerator = np.sum(sims[mask] * (R_values[mask, i] -
user_means[mask]))
        denominator = np.sum(np.abs(sims[mask])) + 1e-8
        preds[i] = user_means[current_user_idx] + numerator / denominator

recs_df = pd.DataFrame({"movieId": R.columns, "pred_rating": preds})
```

```

recs_df = recs_df.dropna().sort_values("pred_rating", ascending=False)
self.paginated_view(
    self.movies.loc[recs_df["movieId"]], title=f"User-based ({metric})"
)

def recommend_item_based(self):
    """Item-based рекомендации"""
    R = self.get_rating_matrix()
    user_ids = R.index.tolist()
    movie_ids = R.columns.tolist()
    current_user_id = -1

    current_user_ratings = pd.Series(
        [np.nan] * len(movie_ids), index=movie_ids, dtype=float
    )
    for movie_id in self.rates.index:
        if movie_id in movie_ids:
            current_user_ratings[movie_id] = self.rates.loc[movie_id,
"rating"]

    R = pd.concat(
        [R, pd.DataFrame([current_user_ratings], index=[current_user_id])]
    )
    current_user_idx = R.index.get_loc(current_user_id)

    metric = Prompt.ask(
        "Выберите метрику сходства [pearson/jaccard/lp/otiai]",
        choices=["pearson", "jaccard", "lp", "otiai"],
        default="pearson",
    )

    R_values = R.values.astype(np.float64)
    n_items = R_values.shape[1]
    item_means = np.nanmean(R_values, axis=0)
    preds = np.full(n_items, np.nan)

    for i in range(n_items):
        if not np.isnan(R_values[current_user_idx, i]):
            continue
        rated_mask = ~np.isnan(R_values[current_user_idx, :])
        sims_i = []
        ratings_i = []
        for j in np.where(rated_mask)[0]:
            mask = ~np.isnan(R_values[:, i]) & ~np.isnan(R_values[:, j])
            if np.sum(mask) == 0:
                sim = 0
            else:
                if metric == "pearson":
                    sim = np.corrcoef(R_values[mask, i], R_values[mask,
j))[0, 1]

                elif metric == "lp":
                    sim = -np.linalg.norm(R_values[mask, i] - R_values[mask,
j])

                elif metric == "jaccard":
                    sim = np.sum(
                        (R_values[mask, i] > 0) & (R_values[mask, j] > 0)
                    ) / np.sum((R_values[mask, i] > 0) | (R_values[mask, j]
> 0))

                elif metric == "otiai":
                    sim = np.dot(R_values[mask, i], R_values[mask, j]) / (
                        np.linalg.norm(R_values[mask, i])

```

```

        * np.linalg.norm(R_values[mask, j])
    )
    sims_i.append(sim)
    ratings_i.append(R_values[current_user_idx, j] - item_means[j])
sims_i = np.array(sims_i)
ratings_i = np.array(ratings_i)
if np.sum(np.abs(sims_i)) > 0:
    preds[i] = item_means[i] + np.dot(sims_i, ratings_i) / np.sum(
        np.abs(sims_i)
    )

recs_df = pd.DataFrame({"movieId": R.columns, "pred_rating": preds})
recs_df = recs_df.dropna().sort_values("pred_rating", ascending=False)
self.paginated_view(
    self.movies.loc[recs_df["movieId"]], title=f"Item-based ({metric})"
)

def recommend_markov_chain(self):
    """Улучшенные рекомендации с помощью дискретной цепи Маркова"""
    if self.rates.empty:
        self.console.print(
            Panel("[red]Вы не оценили ни одного фильма![/red]",
title="Ошибка")
        )
        input("Нажмите Enter, чтобы вернуться...")
        return

    if len(self.rates) < 2:
        self.console.print(
            Panel(
                "[yellow]Оцените хотя бы 2 фильма для построения цепи
Маркова[/yellow]",
                title="Недостаточно данных",
            )
        )

        unrated_movies =
self.movies[~self.movies.index.isin(self.rates.index)]
        if not unrated_movies.empty:
            if Confirm.ask("Показать случайные рекомендации вместо цепи
Маркова?"):
                recommendations = unrated_movies.sample(
                    min(20, len(unrated_movies))
                )
                self.paginated_view(recommendations, title="Случайные
рекомендации")
            else:
                self.console.print("[red]Нет фильмов для рекомендаций[/red]")
                input("Нажмите Enter, чтобы вернуться...")
                return

        markov_rec = self.markov_recommender

        steps = Prompt.ask(
            "Количество шагов цепи Маркова", choices=["1", "2", "3"],
default="1"
        )
        steps = int(steps)

        top_k = Prompt.ask("Количество рекомендаций", default="20")
        top_k = int(top_k)

```

```

        recommendations = markov_rec.recommend_movies(steps=steps, top_k=top_k)

        if recommendations.empty:
            self.console.print(
                Panel(
                    "[yellow]Не найдено подходящих рекомендаций. Попробуйте
оценить больше фильмов разных жанров.[/yellow]",
                    title="Рекомендации",
                )
            )

            unrated_movies =
self.movies[~self.movies.index.isin(self.rates.index)]
            if not unrated_movies.empty:
                if Confirm.ask("Показать случайные рекомендации?"):
                    random_recs = unrated_movies.sample(min(20,
len(unrated_movies)))
                    self.paginated_view(random_recs, title="Случайные
рекомендации")
                else:
                    input("Нажмите Enter, чтобы вернуться...")
                    return

            if Confirm.ask("Показать матрицу переходов?"):
                os.system("cls" if os.name == "nt" else "clear")
                markov_rec.show_transition_matrix()
                input("Нажмите Enter, чтобы продолжить...")

            if Confirm.ask("Показать вероятности жанров?"):
                os.system("cls" if os.name == "nt" else "clear")
                self.show_markov_genre_probabilities(steps)
                input("Нажмите Enter, чтобы продолжить...")

            self.console.print(
                Panel(
                    f"
```

### Продолжение Листинга А

```
prob_table.add_column("Жанр", style="cyan")
prob_table.add_column("Вероятность", style="green")
prob_table.add_column("Процент", style="magenta")

for genre, prob in sorted_genres:
    if prob > 0.001:
        percentage = prob * 100
        prob_table.add_row(genre, f"{prob:.3f}", f"{percentage:.1f}%")

self.console.print(prob_table)

rated_movies = self.movies.loc[self.rates.index]
if not rated_movies.empty:
    last_movie = rated_movies.iloc[-1]
    self.console.print(
        f"\n[bold]Последний оцененный фильм:[/bold]"
[cyan]{last_movie['title']}[/cyan]"
    )
    self.console.print(
        f"[bold]Жанры:[/bold] [green]{'',
'.join(last_movie['genres'])}[/green]"
    )

def get_ratings_with_timestamps(self):
    """Получение оценок с временными метками в хронологическом порядке"""
    if self.rates.empty:
        return []

    ratings_list = []
    for movie_id, row in self.rates.iterrows():
        if 'datetime' in row:
            timestamp = row['datetime']
        else:
            timestamp = datetime.now() - timedelta(days=len(ratings_list))

        ratings_list.append((movie_id, row['rating'], timestamp))

    ratings_list.sort(key=lambda x: x[2])
    return ratings_list

def recommend_continuous_markov_chain(self):
    """Рекомендации на основе непрерывной цепи Маркова с временными
метками"""
    if self.rates.empty:
        self.console.print(
            Panel("[red]Вы не оценили ни одного фильма![/red]",
title="Ошибка")
        )
        input("Нажмите Enter, чтобы вернуться...")
        return

    if len(self.rates) < 2:
        self.console.print(
            Panel(
                "[yellow]Оцените хотя бы 2 фильма для построения непрерывной
цепи Маркова[/yellow]",
                title="Недостаточно данных",
            )
        )
    return
```

### Продолжение Листинга А

```
continuous_markov = self.continuous_markov_recommender

time_t = Prompt.ask(
    "Введите время для прогноза (в днях)", default="1.0"
)
try:
    time_t = float(time_t)
except ValueError:
    time_t = 1.0

top_k = Prompt.ask("Количество рекомендаций", default="20")
top_k = int(top_k)

ratings_with_time =
self.continuous_markov_recommender.get_ratings_chronology()
if ratings_with_time:
    self.console.print("\n[bold]Хронология ваших оценок:[/bold]")
    for i, (movie_id, rating, timestamp) in
enumerate(ratings_with_time):
        movie_title = self.movies.loc[movie_id, "title"]
        time_str = timestamp.strftime("%Y-%m-%d %H:%M") if
isinstance(timestamp, datetime) else "недавно"
        self.console.print(f" {i+1}. {movie_title} - оценка {rating} -
{time_str}")

    recommendations = continuous_markov.recommend_movies_continuous(
        time_t=time_t, top_k=top_k
    )

    if recommendations.empty:
        self.console.print(
            Panel(
                "[yellow]Не найдено подходящих рекомендаций[/yellow]",
                title="Рекомендации",
            )
        )
        input("Нажмите Enter, чтобы вернуться...")
        return

    if Confirm.ask("Показать матрицу интенсивностей?"):
        os.system("cls" if os.name == "nt" else "clear")
        continuous_markov.show_intensity_matrix()
        input("Нажмите Enter, чтобы продолжить...")

    if Confirm.ask("Показать вероятности жанров для непрерывной цепи?"):
        os.system("cls" if os.name == "nt" else "clear")
        self.show_continuous_markov_genre_probabilities(time_t)
        input("Нажмите Enter, чтобы продолжить...")

    self.console.print(
        Panel(
            f"🕒 Рекомендации на основе непрерывной цепи Маркова (t={time_t}
дней)",
            title="Непрерывная цепь Маркова",
        )
    )

    self.paginated_view(
        recommendations,
        title=f"Рекомендации (Непрерывная цепь Маркова, t={time_t} дней)"
    )
```

```

def show_continuous_markov_genre_probabilities(self, time_t=1.0):
    """Показать вероятности жанров для непрерывной цепи Маркова"""
    genre_probs =
self.continuous_markov_recommender.get_genre_probabilities_continuous(time_t)

    if not genre_probs:
        self.console.print(
            "[red]Не удалось вычислить вероятности жанров для непрерывной
цепи. [/red]"
        )
        return

    sorted_genres = sorted(genre_probs.items(), key=lambda x: x[1],
reverse=True)

    prob_table = Table(title=f"Вероятности жанров (непрерывная цепь,
t={time_t})")
    prob_table.add_column("Жанр", style="cyan")
    prob_table.add_column("Вероятность", style="green")
    prob_table.add_column("Процент", style="magenta")

    for genre, prob in sorted_genres:
        if prob > 0.001:
            percentage = prob * 100
            prob_table.add_row(genre, f"{prob:.3f}", f"{percentage:.1f}%")

    self.console.print(prob_table)

    rated_movies = self.movies.loc[self.rates.index]
    if not rated_movies.empty:
        last_movie = rated_movies.iloc[-1]
        self.console.print(
            f"\n[bold]Последний оцененный фильм: [/bold]
[cyan]{last_movie['title']} [/cyan]"
        )
        self.console.print(
            f"[bold]Жанры: [/bold] [green]{',
'.join(last_movie['genres'])} [/green]"
        )

    def recommend_svd(self):
        os.system("cls" if os.name == "nt" else "clear")
        self.console.print(f"[bold cyan]🚀 SVD-модель ({self.svd.k} скрытых
факторов) формируется... [/bold cyan]")

        self.svd.build()
        mse, rmse, mae = self.svd.evaluate()

        self.console.print(f"\n[green]MSE: [/green] {mse:.4f}")
        self.console.print(f"[green]RMSE: [/green] {rmse:.4f}")
        self.console.print(f"[green]MAE: [/green] {mae:.4f}\n")

        top_k = Prompt.ask("Количество рекомендаций", default="20")
        top_k = int(top_k)

        recs = self.svd.recommend_for_current_user(top_k=top_k)

        self.paginated_view(recs, title="SVD рекомендации")

    def plot_similarity_metrics(

```

```

        self, strategy: str = "user-based", test_ratio: float = 0.2
    ):
        metrics = ["pearson", "lp", "jaccard", "otiai"]
        results = []

        for metric in metrics:
            rmse, mae = self._compute_metrics_for_plot(metric, strategy,
test_ratio)
            results.append({"metric": metric, "RMSE": rmse, "MAE": mae})

        df = pd.DataFrame(results)

        sns.set_theme(style="whitegrid", font_scale=1.1)
        fig, axes = plt.subplots(1, 2, figsize=(12, 5))

        sns.barplot(x="metric", y="RMSE", data=df, ax=axes[0],
palette="Blues_d")
        axes[0].set_title(f"Сравнение RMSE ({strategy})")
        axes[0].set_xlabel("Метрика близости")
        axes[0].set_ylabel("RMSE")

        sns.barplot(x="metric", y="MAE", data=df, ax=axes[1],
palette="Greens_d")
        axes[1].set_title(f"Сравнение MAE ({strategy})")
        axes[1].set_xlabel("Метрика близости")
        axes[1].set_ylabel("MAE")

        plt.tight_layout()
        plt.show()

    def _compute_metrics_for_plot(self, metric: str, strategy: str, test_ratio:
float):
        R = self.get_rating_matrix().copy().values.astype(float)
        n_users, n_items = R.shape

        rng = np.random.default_rng(42)
        train = R.copy()
        test_mask = ~np.isnan(R) & (rng.random(R.shape) < test_ratio)
        test_true = np.full_like(R, np.nan)
        test_true[test_mask] = R[test_mask]
        train[test_mask] = np.nan

        user_means = np.nanmean(train, axis=1)
        item_means = np.nanmean(train, axis=0)
        preds = np.full_like(R, np.nan)

        for u in range(n_users):
            if strategy == "user-based":
                sims = np.zeros(n_users)
                current = train[u, :]
                for v in range(n_users):
                    if v == u:
                        continue
                    other = train[v, :]
                    mask = ~np.isnan(current) & ~np.isnan(other)
                    if np.sum(mask) == 0:
                        sims[v] = 0
                        continue
                    if metric == "pearson":
                        sims[v] = np.corrcoef(current[mask], other[mask])[0, 1]
                    elif metric == "lp":

```



```

        sims[v] = -np.linalg.norm(current[mask] - other[mask])
    elif metric == "jaccard":
        sims[v] = np.sum(
            (current[mask] > 0) & (other[mask] > 0)
        ) / np.sum((current[mask] > 0) | (other[mask] > 0))
    elif metric == "otiai":
        sims[v] = np.sum(current[mask] * other[mask]) / (
            np.linalg.norm(current[mask]) *
np.linalg.norm(other[mask])
        )
    else:
        sims[v] = 0

    for i in range(n_items):
        if not np.isnan(train[u, i]):
            continue
        mask = ~np.isnan(train[:, i])
        if np.sum(mask) == 0:
            continue
        numerator = np.sum(sims[mask] * (train[mask, i] -
user_means[mask]))
        denominator = np.sum(np.abs(sims[mask])) + 1e-8
        preds[u, i] = user_means[u] + numerator / denominator

    else:
        for i in range(n_items):
            if not np.isnan(train[u, i]):
                continue
            rated_mask = ~np.isnan(train[u, :])
            sims_i = []
            ratings_i = []
            for j in np.where(rated_mask)[0]:
                mask = ~np.isnan(train[:, i]) & ~np.isnan(train[:, j])
                if np.sum(mask) == 0:
                    sim = 0
                else:
                    if metric == "pearson":
                        sim = np.corrcoef(train[mask, i], train[mask,
j])[0, 1]

                    elif metric == "lp":
                        sim = -np.linalg.norm(train[mask, i] -
train[mask, j])

                    elif metric == "jaccard":
                        sim = np.sum(
                            (train[mask, i] > 0) & (train[mask, j] > 0)
                        ) / np.sum((train[mask, i] > 0) | (train[mask,
j] > 0))

                    elif metric == "otiai":
                        sim = np.sum(train[mask, i] * train[mask, j]) /
(
                            np.linalg.norm(train[mask, i])
                            * np.linalg.norm(train[mask, j])
                        )
                else:
                    sim = 0
                sims_i.append(sim)
                ratings_i.append(train[u, j] - item_means[j])
            sims_i = np.array(sims_i)
            ratings_i = np.array(ratings_i)
            if np.sum(np.abs(sims_i)) > 0:
                preds[u, i] = item_means[i] + np.dot(

```

```

        sims_i, ratings_i
    ) / np.sum(np.abs(sims_i))

    mask_eval = ~np.isnan(test_true) & ~np.isnan(preds)
    if np.sum(mask_eval) == 0:
        return np.nan, np.nan

    diff = preds[mask_eval] - test_true[mask_eval]
    rmse = np.sqrt(np.mean(diff**2))
    mae = np.mean(np.abs(diff))
    return rmse, mae

def svd_analyze(self):
    os.system("cls" if os.name == "nt" else "clear")
    self.console.print("[bold cyan]Анализ влияния числа скрытых факторов
SVD[/bold cyan]")

    k_min = int(Prompt.ask("Минимальное число скрытых факторов",
default="2"))
    k_max = int(Prompt.ask("Максимальное число скрытых факторов",
default="50"))

    self.svd.evaluate_over_k(k_min=k_min, k_max=k_max)

    input("\nНажмите Enter, чтобы вернуться в меню...")

def cluster_menu(self):
    os.system("cls" if os.name == "nt" else "clear")
    self.console.print("[bold cyan]Кластеризация пользователей (SVD) [/bold
cyan]\n")

    n_clusters = int(Prompt.ask("Число кластеров", default="4"))

    self.svd.build()
    labels = self.svd.cluster_users(n_clusters=n_clusters)

    self.console.print(f"\n[green]Готово! Найдено {n_clusters}
кластеров. [/green]")
    self.console.print(f"Распределение: {np.bincount(labels)}")

    input("\nНажмите Enter, чтобы вернуться...")

if __name__ == "__main__":
    cinema = MovieLensCinema(r"../ml-latest-small",
enable_markov_visualization=False)
    cinema.menu()
    # cinema.plot_similarity_metrics()

```

## Приложение Б

### Код файла MarkovChain.py

#### Листинг Б – Код файла MarkovChain.py

```
import numpy as np
import os
import shutil
import networkx as nx
import matplotlib.pyplot as plt
from datetime import datetime
from rich.table import Table

class MarkovChainRecommender:
    """Внутренний класс для рекомендаций на основе цепи Маркова"""

    def __init__(self, cinema, auto_visualize=True):
        self.cinema = cinema
        self.transition_matrix = None
        self.genre_states = []
        self.auto_visualize = auto_visualize

        self._clean_markov_directory()

    def _clean_markov_directory(self):
        """Очистка папки markov при запуске программы"""
        try:
            if os.path.exists("markov"):
                shutil.rmtree("markov")
            os.makedirs("markov", exist_ok=True)
        except Exception as e:
            pass

    def reset_matrix(self):
        """Сброс матрицы переходов (вызывается при изменении оценок)"""
        self.transition_matrix = None
        self.genre_states = []

    def build_transition_matrix(self):
        """Построение матрицы переходов между жанрами с корректной нормализацией
        (вариант 1 – честная модель)"""
        if self.cinema.rates.empty:
            return False

        rated_movies = self.cinema.movies.loc[self.cinema.rates.index]

        all_genres = set()
        for genres in rated_movies["genres"]:
            all_genres.update(genres)
        self.genre_states = sorted(all_genres)

        if not self.genre_states:
            return False

        n_genres = len(self.genre_states)
        self.transition_matrix = np.zeros((n_genres, n_genres))
        state_to_idx = {genre: idx for idx, genre in
            enumerate(self.genre_states)}

        rated_indices = list(rated_movies.index)
```

```
    if len(rated_indices) < 2:
        return False

    for i in range(len(rated_indices) - 1):
        current_movie_id = rated_indices[i]
        next_movie_id = rated_indices[i + 1]

        current_genres = rated_movies.loc[current_movie_id, "genres"]
        next_genres = rated_movies.loc[next_movie_id, "genres"]

        if not current_genres or not next_genres:
            continue

        weight = 1.0 / len(next_genres)

        for curr_genre in current_genres:
            if curr_genre not in state_to_idx:
                continue
            curr_idx = state_to_idx[curr_genre]

            for next_genre in next_genres:
                if next_genre not in state_to_idx:
                    continue
                next_idx = state_to_idx[next_genre]
                self.transition_matrix[curr_idx, next_idx] += weight

    row_sums = self.transition_matrix.sum(axis=1)

    for i in range(n_genres):
        if row_sums[i] > 0:
            self.transition_matrix[i] /= row_sums[i]
        else:
            self.transition_matrix[i, i] = 1.0

    if self.auto_visualize:
        self._auto_visualize_markov_chain()
    return True

def get_genre_probabilities(self, steps=1):
    """Получение вероятностей жанров через steps шагов"""
    if self.transition_matrix is None:
        success = self.build_transition_matrix()
        if not success:
            return None

    if self.cinema.rates.empty:
        return None

    rated_movies = self.cinema.movies.loc[self.cinema.rates.index]
    last_movie_id = rated_movies.index[-1]
    last_genres = rated_movies.loc[last_movie_id, "genres"]

    initial_vector = np.zeros(len(self.genre_states))
    genre_count = len(last_genres)

    if genre_count == 0:
        return None

    for genre in last_genres:
        if genre in self.genre_states:
```

```

        idx = self.genre_states.index(genre)
        initial_vector[idx] = 1.0 / genre_count

    current_probs = initial_vector
    for _ in range(steps):
        current_probs = current_probs @ self.transition_matrix

    return dict(zip(self.genre_states, current_probs))

def recommend_movies(self, steps=1, top_k=20, min_probability=0.01):
    """Рекомендация фильмов на основе цепи Маркова"""
    genre_probs = self.get_genre_probabilities(steps)
    if not genre_probs:
        unrated_movies = self.cinema.movies[
            ~self.cinema.movies.index.isin(self.cinema.rates.index)
        ]
        return unrated_movies.sample(min(20, len(unrated_movies)))

    significant_genres = [
        genre for genre, prob in genre_probs.items() if prob >=
min_probability
    ]

    if not significant_genres:
        significant_genres = [max(genre_probs, key=genre_probs.get)]

    candidate_movies = self.cinema.movies[
        ~self.cinema.movies.index.isin(self.cinema.rates.index)
    ].copy()

    if candidate_movies.empty:
        return candidate_movies

    def calculate_score(genres):
        return sum(genre_probs.get(genre, 0) for genre in genres)

    candidate_movies["markov_score"] = candidate_movies["genres"].apply(
        calculate_score
    )

    recommendations = (
        candidate_movies[candidate_movies["markov_score"] > 0]
        .sort_values("markov_score", ascending=False)
        .head(top_k)
    )

    return recommendations.drop(columns=["markov_score"], errors="ignore")

def show_transition_matrix(self):
    """Отображение матрицы переходов"""
    if self.transition_matrix is None:
        success = self.build_transition_matrix()
        if not success:
            self.cinema.console.print(
                "[red]Не удалось построить матрицу переходов[/red]"
            )
        return

    table = Table(title="Матрица переходов между жанрами")
    table.add_column("From/To", style="cyan")

```

```
for genre in self.genre_states:
    table.add_column(genre, style="green", width=10)

for i, from_genre in enumerate(self.genre_states):
    row = [from_genre]
    for j, to_genre in enumerate(self.genre_states):
        prob = self.transition_matrix[i, j]
        if prob > 0:
            row.append(f"{prob:.3f}")
        else:
            row.append("0")
    table.add_row(*row)

self.cinema.console.print(table)

def _auto_visualize_markov_chain(self, steps=1):
    """Автоматическое построение и сохранение графа цепи Маркова с
    финальными настройками"""
    os.makedirs("markov", exist_ok=True)
    if self.transition_matrix is None or len(self.genre_states) == 0:
        return

    G = nx.DiGraph()

    for genre in self.genre_states:
        G.add_node(genre)

    edge_labels = {}

    for i, from_genre in enumerate(self.genre_states):
        for j, to_genre in enumerate(self.genre_states):
            prob = self.transition_matrix[i, j]
            if prob >= 0.001:
                G.add_edge(from_genre, to_genre, weight=prob)
                edge_labels[(from_genre, to_genre)] = f"{prob:.2f}"

    plt.figure(figsize=(18, 14), dpi=120)

    if len(self.genre_states) <= 6:
        pos = nx.circular_layout(G, scale=3.0)
    elif len(self.genre_states) <= 10:
        pos = nx.circular_layout(G, scale=3.5)
    else:
        pos = nx.spring_layout(G, k=4, iterations=150, scale=3)

    node_size = 8000
    nx.draw_networkx_nodes(
        G,
        pos,
        node_size=node_size,
        node_color="lightblue",
        alpha=0.95,
        edgecolors="navy",
        linewidths=3,
        node_shape="o",
    )

    nx.draw_networkx_labels(
        G,
        pos,
        font_size=16,
```

```

        font_weight="bold",
        font_family="DejaVu Sans",
        verticalalignment="center",
        horizontalalignment="center",
    )

    if G.edges():
        edge_weights = [G[u][v]["weight"] for u, v in G.edges()]
        max_weight = max(edge_weights) if edge_weights else 1

        edge_widths = [1.5 + (w / max_weight) * 2.5 for w in edge_weights]
        edge_alphas = [0.4 + (w / max_weight) * 0.6 for w in edge_weights]

        edges = nx.draw_networkx_edges(
            G,
            pos,
            edge_color="darkred",
            width=edge_widths,
            alpha=edge_alphas,
            arrows=True,
            arrowsize=30,
            arrowstyle="-|>",
            connectionstyle="arc3,rad=0.25",
            min_source_margin=20,
            min_target_margin=25,
            node_size=node_size,
            ax=plt.gca(),
        )

        nx.draw_networkx_edge_labels(
            G,
            pos,
            edge_labels=edge_labels,
            font_size=12,
            font_weight="bold",
            font_family="DejaVu Sans",
            label_pos=0.18,
            verticalalignment="center",
            horizontalalignment="center",
            bbox=dict(
                boxstyle="round,pad=0.3",
                facecolor="white",
                alpha=0.9,
                edgecolor="lightgray",
                linewidth=1.5,
            ),
        )

    title_info = f"Цепь Маркова: переходы между жанрами (k={steps})\n"
    title_info += f"Узлы: {len(G.nodes())} | Переходы: {len(G.edges())} | "
    title_info += f"Сгенерировано: {datetime.now().strftime('%d.%m.%Y %H:%M')}"

    plt.title(
        title_info,
        fontsize=18,
        fontweight="bold",
        pad=30,
        loc="center",
        fontfamily="DejaVu Sans",
    )

```

*Окончание Листинга Б*

```
plt.axis("off")
plt.tight_layout(pad=4.0)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f"markov/markov_chain_k{steps}_{timestamp}.png"
plt.savefig(
    filename, dpi=150, bbox_inches="tight", facecolor="white",
    edgecolor="none"
)
plt.close()
```



## Приложение В

### Код файла ContinuousMarkovChain.py

#### Листинг В – Код файла ContinuousMarkovChain.py

```
import numpy as np
import pandas as pd
from scipy.linalg import expm
from rich.table import Table
import os
import matplotlib.pyplot as plt
import networkx as nx
from datetime import datetime, timedelta

class ContinuousMarkovRecommender:
    """Исправленная реализация непрерывной цепи Маркова с реальными временными метками"""

    def __init__(self, cinema, auto_visualize=True):
        self.cinema = cinema
        self.intensity_matrix = None
        self.genre_states = []
        self.auto_visualize = auto_visualize
        self.time_unit = 1.0

    def reset_matrix(self):
        """Сброс матрицы интенсивностей"""
        self.intensity_matrix = None
        self.genre_states = []

    def build_intensity_matrix(self):
        """Построение матрицы интенсивностей с использованием реальных данных"""
        if self.cinema.rates.empty or len(self.cinema.rates) < 2:
            return False

        ratings_chronology = self.get_ratings_chronology()
        if len(ratings_chronology) < 2:
            return False

        all_genres = set()
        for movie_id, rating, time_info in ratings_chronology:
            movie_genres = self.cinema.movies.loc[movie_id, "genres"]
            all_genres.update(movie_genres)
        self.genre_states = sorted(all_genres)

        if not self.genre_states:
            return False

        n_genres = len(self.genre_states)
        state_to_idx = {genre: idx for idx, genre in
            enumerate(self.genre_states)}

        time_in_state = {genre: 0.0 for genre in self.genre_states}
        transition_counts = np.zeros((n_genres, n_genres))

        for i in range(len(ratings_chronology) - 1):
            current_movie_id, current_rating, current_time =
            ratings_chronology[i]
            next_movie_id, next_rating, next_time = ratings_chronology[i + 1]

            current_genres = self.cinema.movies.loc[current_movie_id, "genres"]
```

```

        next_genres = self.cinema.movies.loc[next_movie_id, "genres"]

        if not current_genres or not next_genres:
            continue

        time_interval = self.calculate_time_interval(current_time,
next_time)

        for genre in current_genres:
            if genre in state_to_idx:
                time_in_state[genre] += time_interval / len(current_genres)

        for curr_genre in current_genres:
            if curr_genre not in state_to_idx:
                continue
            curr_idx = state_to_idx[curr_genre]

            for next_genre in next_genres:
                if next_genre not in state_to_idx:
                    continue
                next_idx = state_to_idx[next_genre]
                weight = 1.0 / (len(current_genres) * len(next_genres))
                transition_counts[curr_idx, next_idx] += weight

        self.intensity_matrix = np.zeros((n_genres, n_genres))

        for i, genre_i in enumerate(self.genre_states):
            total_time = time_in_state[genre_i]
            if total_time > 0:
                for j, genre_j in enumerate(self.genre_states):
                    if i != j and transition_counts[i, j] > 0:
                        self.intensity_matrix[i, j] = (
                            transition_counts[i, j] / total_time
                        )

                self.intensity_matrix[i, i] = -np.sum(self.intensity_matrix[i])

        symmetric_matrix = (self.intensity_matrix + self.intensity_matrix.T) / 2

        for i in range(n_genres):
            for j in range(n_genres):
                if i != j:
                    self.intensity_matrix[i, j] = max(symmetric_matrix[i, j], 0)

                self.intensity_matrix[i, i] = -np.sum(self.intensity_matrix[i])

        min_intensity = 0.01
        for i in range(n_genres):
            if abs(self.intensity_matrix[i, i]) < min_intensity:
                for j in range(n_genres):
                    if i != j:
                        self.intensity_matrix[i, j] += min_intensity / (n_genres
- 1)

                self.intensity_matrix[i, i] = -np.sum(self.intensity_matrix[i])

        if self.auto_visualize:
            self._auto_visualize_markov_chain()

        return True

    def get_ratings_chronology(self, random_intervals=True):

```

### Продолжение Листинга В

```
"""Создание хронологии оценок с реалистичными случайными интервалами"""
if self.cinema.rates.empty:
    return []

ratings_list = []
current_time = datetime.now()

prev_time = current_time - timedelta(days=len(self.cinema.rates) * 2)

for i, (movie_id, row) in enumerate(self.cinema.rates.iterrows()):
    if random_intervals:
        interval_days = np.random.exponential(scale=2.0)
    else:
        interval_days = 1.0

    timestamp = prev_time + timedelta(days=interval_days)
    ratings_list.append((movie_id, row["rating"], timestamp))
    prev_time = timestamp

return sorted(ratings_list, key=lambda x: x[2])

def calculate_time_interval(self, time1, time2):
    """Вычисление временного интервала между двумя оценками"""
    time_diff = abs((time2 - time1).total_seconds()) / (24 * 3600)
    return max(time_diff, 0.1)

def get_genre_probabilities_continuous(self, time_t):
    """Получение вероятностей жанров в момент времени t с нормализацией"""
    if self.intensity_matrix is None:
        success = self.build_intensity_matrix()
        if not success:
            return None

    if self.cinema.rates.empty:
        return None

    ratings_chronology = self.get_ratings_chronology()
    if not ratings_chronology:
        return None

    last_movie_id, last_rating, last_time = ratings_chronology[-1]
    last_genres = self.cinema.movies.loc[last_movie_id, "genres"]

    initial_vector = np.zeros(len(self.genre_states))
    genre_count = len(last_genres)

    if genre_count == 0:
        return None

    for genre in last_genres:
        if genre in self.genre_states:
            idx = self.genre_states.index(genre)
            initial_vector[idx] = 1.0 / genre_count

    try:
        transition_matrix = expm(self.intensity_matrix * time_t)
        current_probs = initial_vector @ transition_matrix

        prob_sum = np.sum(current_probs)
        if prob_sum > 0:
            current_probs = current_probs / prob_sum
```

```

        else:
            current_probs = np.ones_like(current_probs) / len(current_probs)

    except Exception as e:
        print(f"Ошибка вычисления матричной экспоненты: {e}")
        return None

    return dict(zip(self.genre_states, current_probs))

def recommend_movies_continuous(self, time_t=1.0, top_k=20):
    """Улучшенные рекомендации, чувствительные к времени"""
    genre_probs = self.get_genre_probabilities_continuous(time_t)
    if not genre_probs:
        unrated_movies = self.cinema.movies[
            ~self.cinema.movies.index.isin(self.cinema.rates.index)
        ]
        return unrated_movies.sample(min(top_k, len(unrated_movies)))

    candidate_movies = self.cinema.movies[
        ~self.cinema.movies.index.isin(self.cinema.rates.index)
    ].copy()

    if candidate_movies.empty:
        return candidate_movies

    def calculate_score(genres, time_factor=time_t):
        if not genres:
            return 0

        base_score = sum(genre_probs.get(genre, 0) for genre in genres)

        diversity_bonus = len(set(genres) & set(genre_probs.keys())) /
len(genres)

        time_weight = min(time_t / 5.0, 1.0)
        final_score = base_score * (1 - time_weight) + diversity_bonus *
time_weight

        return final_score

    candidate_movies["continuous_score"] = candidate_movies["genres"].apply(
        lambda genres: calculate_score(genres, time_t)
    )

    max_score = candidate_movies["continuous_score"].max()
    if max_score > 0:
        candidate_movies["continuous_score"] = (
            candidate_movies["continuous_score"] / max_score
        )

    recommendations = (
        candidate_movies[candidate_movies["continuous_score"] > 0.01]
        .sort_values("continuous_score", ascending=False)
        .head(top_k)
    )

    return recommendations.drop(columns=["continuous_score"],
errors="ignore")

def show_intensity_matrix(self):
    """Отображение матрицы интенсивностей с дополнительной информацией"""

```

```
        if self.intensity_matrix is None:
            success = self.build_intensity_matrix()
            if not success:
                self.cinema.console.print(
                    "[red]Не удалось построить матрицу интенсивностей[/red]"
                )
            return

        table = Table(
            title="Матрица интенсивностей переходов (непрерывная цепь Маркова)"
        )
        table.add_column("From/To", style="cyan")

        for genre in self.genre_states:
            table.add_column(genre, style="green", width=10)

        for i, from_genre in enumerate(self.genre_states):
            row = [from_genre]
            for j, to_genre in enumerate(self.genre_states):
                intensity = self.intensity_matrix[i, j]
                if abs(intensity) > 0.001:
                    if i == j:
                        row.append(f"[red]{intensity:+.3f}[/red]")
                    else:
                        row.append(f"{intensity:+.3f}")
                else:
                    row.append("0.000")
            table.add_row(*row)

        self.cinema.console.print(table)

        self.cinema.console.print(
            f"\n[bold]Размер матрицы:[/bold] {len(self.genre_states)}x{len(self.genre_states)}"
        )
        self.cinema.console.print(
            f"[bold]Количество жанров:[/bold] {len(self.genre_states)}"
        )

    def _auto_visualize_markov_chain(self, time_t=1.0):
        """Автоматическая визуализация и сохранение графа непрерывной цепи
        Маркова"""
        os.makedirs("markov", exist_ok=True)
        if self.intensity_matrix is None or len(self.genre_states) == 0:
            return

        G = nx.DiGraph()

        for genre in self.genre_states:
            G.add_node(genre)

        edge_labels = {}

        for i, from_genre in enumerate(self.genre_states):
            for j, to_genre in enumerate(self.genre_states):
                intensity = self.intensity_matrix[i, j]
                if intensity > 0.001:
                    G.add_edge(from_genre, to_genre, weight=intensity)
                    edge_labels[(from_genre, to_genre)] = f"{intensity:.2f}"

        plt.figure(figsize=(18, 14), dpi=120)
```

```
if len(self.genre_states) <= 6:
    pos = nx.circular_layout(G, scale=3.0)
elif len(self.genre_states) <= 10:
    pos = nx.circular_layout(G, scale=3.5)
else:
    pos = nx.spring_layout(G, k=4, iterations=150, scale=3)

node_size = 8000
nx.draw_networkx_nodes(
    G,
    pos,
    node_size=node_size,
    node_color="lightblue",
    alpha=0.95,
    edgecolors="navy",
    linewidths=3,
    node_shape="o",
)

nx.draw_networkx_labels(
    G,
    pos,
    font_size=16,
    font_weight="bold",
    font_family="DejaVu Sans",
    verticalalignment="center",
    horizontalalignment="center",
)

if G.edges():
    edge_weights = [G[u][v]["weight"] for u, v in G.edges()]
    max_weight = max(edge_weights) if edge_weights else 1

    edge_widths = [1.5 + (w / max_weight) * 2.5 for w in edge_weights]
    edge_alphas = [0.4 + (w / max_weight) * 0.6 for w in edge_weights]

    edges = nx.draw_networkx_edges(
        G,
        pos,
        edge_color="darkred",
        width=edge_widths,
        alpha=edge_alphas,
        arrows=True,
        arrowsize=30,
        arrowstyle="-|>",
        connectionstyle="arc3,rad=0.25",
        min_source_margin=20,
        min_target_margin=25,
        node_size=node_size,
        ax=plt.gca(),
    )

    nx.draw_networkx_edge_labels(
        G,
        pos,
        edge_labels=edge_labels,
        font_size=12,
        font_weight="bold",
        font_family="DejaVu Sans",
        label_pos=0.18,
```

### Окончание Листинга В

```
        verticalalignment="center",
        horizontalalignment="center",
        bbox=dict(
            boxstyle="round,pad=0.3",
            facecolor="white",
            alpha=0.9,
            edgecolor="lightgray",
            linewidth=1.5,
        ),
    ),

    title_info = f"Непрерывная цепь Маркова: интенсивности переходов
(t={time_t})\n"
    title_info += f"Узлы: {len(G.nodes())} | Переходы: {len(G.edges())} | "
    title_info += f"Сгенерировано: {datetime.now().strftime('%d.%m.%Y
%H:%M')}"

    plt.title(
        title_info,
        fontsize=18,
        fontweight="bold",
        pad=30,
        loc="center",
        fontfamily="DejaVu Sans",
    )

    plt.axis("off")
    plt.tight_layout(pad=4.0)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"markov/continuous_markov_chain_t{time_t}_{timestamp}.png"
    plt.savefig(
        filename, dpi=150, bbox_inches="tight", facecolor="white",
edgecolor="none"
    )
    plt.close()
```

## Приложение Г

### Код файла SVDRecommender.py

Листинг Г – Код файла SVDRecommender.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    root_mean_squared_error,
)
from sklearn.decomposition import PCA
from sklearn.mixture import GaussianMixture
from sklearn.cluster import AgglomerativeClustering

class SVDRecommender:
    def __init__(self, cinema, k=20):
        self.cinema = cinema
        self.k = k
        self.U = None
        self.D = None
        self.Vt = None
        self.pred_matrix = None

    def build(self):
        """SVD по матрице рейтингов"""
        R = self.cinema.get_rating_matrix()

        R_filled = R.fillna(0).values

        U, D, Vt = np.linalg.svd(R_filled, full_matrices=False)

        U_k = U[:, : self.k]
        D_k = np.diag(D[: self.k])
        Vt_k = Vt[:, : self.k, :]

        self.U = U_k
        self.D = D_k
        self.Vt = Vt_k

        self.pred_matrix = np.dot(U_k, np.dot(D_k, Vt_k))

        return self.pred_matrix

    def evaluate(self):
        """Вычисление RMSE, MSE, MAE по реальным оценкам"""
        ratings = self.cinema.ratings.reset_index()
        movie_ids = self.cinema.get_rating_matrix().columns.tolist()
        user_ids = self.cinema.get_rating_matrix().index.tolist()

        preds = []
        reals = []

        for _, row in ratings.iterrows():
            uid = row["userId"]
            mid = row["movieId"]

            if uid not in user_ids or mid not in movie_ids:
                continue
```



```

        u_idx = user_ids.index(uid)
        m_idx = movie_ids.index(mid)
        pred = self.pred_matrix[u_idx, m_idx]

        preds.append(pred)
        reals.append(row["rating"])

    mse = mean_squared_error(reals, preds)
    rmse = root_mean_squared_error(reals, preds)
    mae = mean_absolute_error(reals, preds)

    return mse, rmse, mae

def recommend_for_current_user(self, top_k=20):
    """Рекомендации для текущего пользователя"""
    R = self.cinema.get_rating_matrix()
    movie_ids = R.columns.tolist()

    current_user_id = -1
    current_ratings = pd.Series([np.nan] * len(movie_ids), index=movie_ids)

    for mid in self.cinema.rates.index:
        if mid in movie_ids:
            current_ratings[mid] = self.cinema.rates.loc[mid, "rating"]

    R_ext = pd.concat(
        [R, pd.DataFrame([current_ratings], index=[current_user_id])]
    ).fillna(0)

    u_vec = np.dot(
        np.dot(R_ext.loc[current_user_id].values, self.Vt.T),
        np.linalg.inv(self.D)
    )

    preds = np.dot(u_vec, self.Vt)

    result = pd.DataFrame({"movieId": movie_ids, "pred": preds})

    result = result[~result["movieId"].isin(self.cinema.rates.index)]

    result = result.sort_values("pred", ascending=False).head(top_k)

    return self.cinema.movies.loc[result["movieId"]]

def evaluate_over_k(self, k_min=2, k_max=50):
    R = self.cinema.get_rating_matrix()
    R_filled = R.fillna(0).values
    ratings = self.cinema.ratings.reset_index()

    movie_ids = R.columns.tolist()
    user_ids = R.index.tolist()

    ks = []
    mses = []
    rmses = []
    maes = []

    for k in range(k_min, k_max + 1):
        self.cinema.console.print(f"Построение SVD для k={k}: ",
            highlight=False, end="")

```

```

        U, D, Vt = np.linalg.svd(R_filled, full_matrices=False)

        U_k = U[:, :k]
        D_k = np.diag(D[:k])
        Vt_k = Vt[:k, :]

        pred_matrix = np.dot(U_k, np.dot(D_k, Vt_k))

        preds = []
        reals = []

        for _, row in ratings.iterrows():
            uid = row["userId"]
            mid = row["movieId"]

            if uid not in user_ids or mid not in movie_ids:
                continue

            u_idx = user_ids.index(uid)
            m_idx = movie_ids.index(mid)

            preds.append(pred_matrix[u_idx, m_idx])
            reals.append(row["rating"])

        mse = mean_squared_error(reals, preds)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(reals, preds)

        self.cinema.console.print(f"[green]MSE:[/green] {mse:.4f},
[green]RMSE:[/green] {rmse:.4f}, [green]MAE:[/green] {mae:.4f}")

        ks.append(k)
        mses.append(mse)
        rmse.append(rmse)
        maes.append(mae)

        plt.figure(figsize=(10, 6))
        plt.plot(ks, mses, label="MSE")
        plt.plot(ks, rmse, label="RMSE")
        plt.plot(ks, maes, label="MAE")
        plt.xlabel("Число скрытых факторов")
        plt.ylabel("Значение метрики")
        plt.title("Зависимость MSE, RMSE, MAE от числа факторов k")
        plt.legend()
        plt.grid(True)
        plt.show()

    def cluster_users(self, n_clusters=4):
        if self.U is None:
            raise ValueError("Сначала постройте SVD с помощью build().")

        U_k = self.U

        pca = PCA(n_components=2)
        U_2d = pca.fit_transform(U_k)

        model = AgglomerativeClustering(n_clusters=n_clusters)
        labels = model.fit_predict(U_2d)

        plt.figure(figsize=(8, 6))

```

*Окончание Листинга Г*

```
plt.scatter(U_2d[:, 0], U_2d[:, 1], c=labels, cmap="tab10")
plt.title(f"Кластеризация пользователей")
plt.xlabel("PCA component 1")
plt.ylabel("PCA component 2")
plt.grid(True)
plt.show()

return labels
```